

Uniwersytet Jagielloński
Wydział Zarządzania i Komunikacji Społecznej
INSTYTUT STUDIÓW INFORMACYJNYCH

Kierunek: elektroniczne przetwarzanie informacji

Przewodnik seminaryjny

Praca licencjacka i projekt dyplomowy na kierunku EPI

dr Marek Deja

Kraków 2027

Spis treści

Wprowadzenie	7
0.1 Dla kogo i po co	7
0.2 Jak czytać	7
0.3 Dokumenty rozstrzygające	8
0.4 Dwie ścieżki pracy	8
1 LaTeX od podstaw	9
1.1 Jak myśleć o LaTeX-u	9
1.2 Poziom pierwszy: tekst i struktura	9
1.2.1 Anatomia dokumentu	9
1.2.2 Akapity i spacje	10
1.2.3 Polskie znaki i typografia	10
1.2.4 Rozdziały i podrozdziały	11
1.2.5 Etykiety i odsyłacze	11
1.2.6 Cytowania	12
1.2.7 Polecenia własne wzoru	12
1.3 Poziom drugi: tabele, rysunki, wzory	13
1.3.1 Listy i cytaty	13
1.3.2 Tabele	13
1.3.3 Rysunki	13
1.3.4 Wzory	14
1.3.5 Listingi kodu	15
1.4 Poziom trzeci: przy redakcji	16
1.4.1 Praca w wielu plikach	16
1.4.2 Kontrola typografii	16
1.4.3 Sekwencja kompilacji	16
1.5 Diagnostyka	17
1.5.1 Jak czytać błąd	17
1.5.2 Katalog błędów	17
1.5.3 Gdy nic nie pomaga	17
2 Bibliografia: plik BibTeX i Zotero	19
2.1 Anatomia wpisu	19

2.1.1	Klucze	19
2.1.2	Pola nazwisk	20
2.2	Typy wpisów	20
2.2.1	Książka i praca zbiorowa	21
2.2.2	Rozdział w pracy zbiorowej	21
2.2.3	Artykuł	21
2.2.4	Hasło w wydawnictwie informacyjnym	22
2.2.5	Dokumenty sieciowe	22
2.2.6	Pozycje szczególne	22
2.3	Zestawienie pól	24
2.4	Zotero	24
2.4.1	Konfiguracja	24
2.4.2	Odpowiedniki typów	24
2.4.3	Poprawki po imporcie	25
2.5	Cytowania w tekście	25
2.6	Cytowanie oprogramowania i danych	25
2.7	Kontrola	26
2.8	Wariant APA	26
3	Overleaf i współpraca	28
3.1	Ścieżka studenta	28
3.1.1	Konto i projekt	28
3.1.2	Ustawienia	28
3.1.3	Co jest czym w projekcie	29
3.1.4	Kompilacja i kopie	29
3.1.5	Udostępnienie i oddanie	29
3.2	Plik bibliograficzny a Zotero	29
3.3	Ścieżka promotora	30
3.3.1	Rozdanie wzoru grupie	30
3.3.2	Przegląd prac	30
3.3.3	Kontrola formalna	31
3.4	Praca lokalna w Positronie	31
3.5	Problemy specyficzne dla Overleaf	32
4	Jak napisać pracę	33
4.1	Wprowadzenie	33
4.1.1	Co musi zawierać	33
4.1.2	Typowe błędy	34
4.2	Rozdział pierwszy: podstawy metodyczne	35
4.2.1	Zasada nadrzędna	35

4.2.2	Co musi zawierać	35
4.2.3	Skąd brać treść	35
4.3	Rozdział drugi: implementacja i architektura	36
4.3.1	Co musi zawierać	36
4.3.2	Jak pisać o pracy z agentem	37
4.4	Rozdział trzeci: studium przypadku	38
4.4.1	Co musi zawierać	38
4.5	Podsumowanie	39
4.6	Elementy końcowe	39
4.6.1	Wykaz źródeł	39
4.6.2	Bibliografia	40
4.6.3	Spis ilustracji i indeks nazwisk	40
4.6.4	Aneksy	40
4.7	W jakiej kolejności pisać	40
5	Warsztat pisania akademickiego	42
5.1	Pytanie analityczne	42
5.1.1	Jakie pytanie nadaje się na fundament pracy	42
5.1.2	Skąd brać pytania: punkty napięcia	42
5.1.3	Napięcia właściwe dla pracy o oprogramowaniu badawczym	43
5.1.4	Jak formułować	43
5.2	Od pytania do tezy	43
5.3	Wejście w dyskusję akademicką	44
5.3.1	Krok w przód	44
5.3.2	Krok w tył	44
5.3.3	Kwestionowanie	44
5.4	Zapożyczanie i rozwijanie teorii	44
5.5	Czytanie artykułu metodycznego	45
5.6	Notatki i zarządzanie źródłami	45
5.7	Styl naukowy	46
5.7.1	Terminologia	46
5.7.2	Akapit	46
5.7.3	Rejestr	46
5.7.4	Precyzja	47
5.8	Rzetelność	47
5.8.1	Oryginalność	47
5.8.2	Kompilacja	47
5.8.3	Granica między własnym a cudzym	47
5.8.4	Stan badań w układzie problemowym	48

5.9	Redakcja i korekta	48
5.10	Harmonogram	48
6	Wymogi formalne	50
6.1	Skład	50
6.2	Objętość	50
6.3	Struktura	51
6.3.1	Strona informacyjna	51
6.4	Cytowanie	51
6.5	Przypisy i bibliografia	52
6.6	Materiał ilustracyjny	52
6.7	Zapis w tekście	53
6.8	Wymogi projektu dyplomowego	53
6.9	Kryteria oceny	54
6.10	Lista kontrolna przed oddaniem	54
7	Praca z agentem programistycznym	56
7.1	Środowisko	56
7.1.1	Positron	56
7.1.2	Wybór agenta	57
7.1.3	Podpowiadanie w edytorze to co innego	57
7.1.4	Konfiguracja repozytorium	57
7.2	Kolejność pracy	58
7.2.1	Cztery kroki	58
7.2.2	Dlaczego odwrotna kolejność zawodzi	58
7.3	Pętla naprawcza	58
7.3.1	Kiedy przerwać	58
7.3.2	Miejsca, w których agent zgaduje	59
7.4	Szablony poleceń	59
7.4.1	Wzór na kod	60
7.4.2	Dane, na których metoda nie ma sensu	60
7.4.3	Diagnostyka zamiast zgadywania	60
7.4.4	Dokumentacja ze specyfikacji	61
7.5	Rejestr poleceń	61
7.6	Co robisz sam	62
8	Jawność i odpowiedzialność	63
8.1	Dwa różne obszary	63
8.1.1	Kod aplikacji	63
8.1.2	Tekst pracy	63
8.1.3	Bezwzględny zakaz	64

8.2	Oświadczenie	64
8.3	Aneks z wykazem poleceń	65
8.4	Przygotowanie do obrony	65
8.5	Granica, której nie przekraczamy	65
	Bibliografia	67
	Spis ilustracji	68

Wprowadzenie

Przewodnik obejmuje cały cykl seminarium: od pierwszego uruchomienia wzoru, przez budowę pakietu i pisanie kolejnych rozdziałów, po kontrolę formalną przed oddaniem pracy.

Praca dyplomowa na kierunku elektroniczne przetwarzanie informacji składa się z dwóch ściśle powiązanych części: projektu dyplomowego oraz pracy licencjackiej, która ten projekt opisuje. Zaliczenie seminarium następuje dopiero po przyjęciu projektu (Standardy EPI). Twoim projektem jest pakiet języka R implementujący algorytm z artykułu metodycznego wraz z serwisem, który go udostępnia.

0.1 Dla kogo i po co

Przewodnik zakłada, że znasz podstawy statystyki i programowania w R (R Core Team 2026), a nie znasz LaTeX-a ani inżynierii oprogramowania. Wszystkiego, co potrzebne, nauczysz się tutaj od zera.

Dokument, który czytasz, jest jednocześnie instrukcją i przykładem. Złożono go tą samą klasą, z której korzystasz przy pisaniu pracy, i tym samym stylem bibliograficznym. Każdy element opisany w rozdziale pierwszym – tabela z tytułem nad nią, wzór z etykietą, listing kodu, cytowanie w nawiasie, spis ilustracji – występuje dalej w praktyce. Kiedy nie pamiętasz zapisu, zajrzyj do źródła tego przewodnika: leży w tym samym repozytorium.

0.2 Jak czytać

Rozdziały pierwszy, drugi i trzeci dotyczą narzędzi. Przeczytaj je na początku seminarium, zanim napiszesz pierwsze zdanie pracy. Rozdział pierwszy wystarczy przerobić do połowy, żeby zacząć – reszta przyda się później.

Rozdziały czwarty, piąty i szósty dotyczą pisania. Do rozdziału czwartego wracaj przy każdej kolejnej części pracy. Rozdział piąty przeczytaj raz, na samym początku, bo pytanie badawcze stawia się przed pisaniem, a nie po nim. Rozdział szósty służy jako lista kontrolna przed oddaniem.

Rozdziały siódmy i ósmy dotyczą pracy z narzędziem wspomagającym pisanie kodu. Siódmy opisuje metodę, ósmy granice dozwolonego użycia i sposób jego dokumentowania. Oba przeczytaj **zanim wydasz pierwsze polecenie**, a nie wtedy, gdy pakiet będzie już gotowy: część wymagań dotyczy rzeczy, których nie da się odtworzyć po fakcie.

Tabela 1. Zawartość przewodnika

Rozdział	Temat	Kiedy czytać
1	LaTeX	na początku, pierwsze dwa poziomy
2	Bibliografia i Zotero	przed pierwszą lekturą
3	Overleaf i współpraca	przy zakładaniu projektu
4	Jak napisać pracę	przy pisaniu każdej części
5	Warsztat akademicki	raz, na samym początku
6	Wymogi formalne	przed oddaniem
7	Praca z agentem programistycznym	przed pierwszym poleceniem
8	Jawność i odpowiedzialność	na początku i przed oddaniem

0.3 Dokumenty rozstrzygające

Wymagania wobec pracy określają dwa dokumenty Instytutu. **Standardy prac dyplomowych EPI** (Standardy EPI) rozstrzygają o układzie pracy i o wymogach wobec projektu dyplomowego. **Instrukcja ISI** (Instrukcja ISI) rozstrzyga o składzie, przypisach i opisach bibliograficznych. Tam, gdzie dokumenty się rozchodzą, w przewodniku wskazany jest ten, który obowiązuje.

Wymagania te są we wzorze zrealizowane automatycznie. Nie zmieniaj ustawień składu – rozbieżność z wymogami obciąża pracę, a nie wzór.

0.4 Dwie ścieżki pracy

Pracę można pisać na dwa sposoby i przewodnik opisuje oba.

Ścieżka przeglądarkowa. Piszesz na uczelnianej instalacji Overleaf. Nie instalujesz niczego, kompilacja dzieje się na serwerze, promotor widzi tę samą wersję co Ty. Jest to ścieżka domyślna i wystarczająca do napisania całej pracy.

Ścieżka lokalna. Piszesz u siebie, w Positronie, mając zainstalowaną dystrybucję $\text{\TeX}$ -a i R. Zyskujesz to, że praca, pakiet i wykresy są w jednym miejscu, oraz dostęp do skryptów kontrolnych, których na Overleaf uruchomić się nie da.

Różnica jest istotna w kilku miejscach: przy kompilacji, przy czyszczeniu plików pomocniczych i przy kontroli objętości. Wszędzie tam, gdzie postępowanie się rozchodzi, akapity są oznaczone słowami **Lokalnie** i **Na Overleaf**. Tam, gdzie takiego oznaczenia nie ma, opis dotyczy obu ścieżek.

Najczęstszy układ w praktyce jest mieszany: pakiet i wykresy powstają lokalnie, tekst pracy na Overleaf. Wtedy stosujesz oznaczenie właściwe dla czynności, a nie dla całej pracy.

1 LaTeX od podstaw

W rozdziale poznajesz LaTeX-a na wzorze, którym piszesz pracę, a nie w oderwaniu od niego. Wszystkie przykłady działają bez dodatkowej konfiguracji, bo ustawienia składu są w *klasie dokumentu*, czyli w pliku, w którym zapisany jest cały wygląd pracy. Twoja klasa to `epi-praca.cls` i nie musisz do niej zaglądać.

Podziel naukę na trzy wieczory. Po pierwszym umiesz złożyć tekst z rozdziałami i cytowaniami. Po drugim – tabele, rysunki i wzory. Trzeci przyda się dopiero przy redakcji.

1.1 Jak myśleć o LaTeX-u

W edytorze tekstu zaznaczasz fragment i nadajesz mu wygląd. W LaTeX-u **opisujesz, czym fragment jest**, a wygląd wynika z tego automatycznie. Piszesz „to jest tytuł rozdziału”, a nie „to ma być osiemnaście punktów, wytłuszczone, wyśrodkowane”.

Ma to trzy konsekwencje, które w pracy licencjackiej są warte więcej niż koszt nauki. Skład jest zgodny z wymogami Instytutu z definicji, a nie dlatego, że pamiętałeś o ustawieniu interlinii w każdym akapicie. Spis treści, spis ilustracji, bibliografia i indeks nazwisk powstają same z tego, co napisałeś w tekście – nie da się mieć w spisie treści rozdziału, którego nie ma. Wzory matematyczne są zapisem, a nie obrazkiem, co przy pracy pełnej wzorów z artykułu metodycznego przestaje być wygodą, a staje się warunkiem wykonalności.

Kosztem jest to, że pliku źródłowego się nie ogląda – trzeba go najpierw *skompilować*, czyli przetworzyć w gotowy dokument PDF. Do tego przyzwyczaisz się w pierwszy dzień.

1.2 Poziom pierwszy: tekst i struktura

1.2.1 Anatomia dokumentu

Praca składa się z pliku głównego i plików rozdziałów.

```
\documentclass{epi-praca}           % klasa: cały skład

\addbibresource{bibliografia/literatura.bib}
\autor{Jan Kowalski}               % metadane
\tytul{Tytuł pracy}

\begin{document}                   % początek treści
\stronatytułowa
\tableofcontents
```

```
\input{rozdzialy/00-wprowadzenie}  
\end{document} % koniec tresci
```

Wszystko przed `\begin{document}` to *preambuła*, czyli ustawienia. Wszystko między `\begin{document}` a `\end{document}` to treść. W Twojej pracy preambuła jest już gotowa; uzupełniasz w niej wyłącznie metadane.

Polecenie zaczyna się od ukośnika wstecznego i mówi, co ma zostać zrobione. To, na czym polecenie działa, nazywa się *argumentem*. Argumenty obowiązkowe są w nawiasach klamrowych, opcjonalne w kwadratowych: w zapisie `\includegraphics[width=5cm]{wykres.png}` nazwa pliku jest argumentem obowiązkowym, a szerokość opcjonalnym.

Środowisko obejmuje cały fragment tekstu i ma jawny początek oraz koniec. Otwiera je `\begin` z nazwą, zamyka `\end` z tą samą nazwą, a wszystko pomiędzy składa się według reguł tego środowiska – jak `quote` dla cytatu blokowego albo `tabular` dla tabeli. Środowiskiem jest też sam dokument, o czym mówi para poleceń z przykładu wyżej.

1.2.2 Akapity i spacje

Nowy akapit zaczyna się po **pustej linii**. Pojedyncze złamanie wiersza w pliku źródłowym nie znaczy nic – LaTeX i tak złoży tekst od nowa. Wielokrotne spacje redukują się do jednej. Nie wymuszaj odstępów spacjami ani pustymi liniami; od tego są polecenia, a w Twojej pracy odstępy ustawia klasa.

W seminarium obowiązuje jedna zasada zapisu źródła: **jeden akapit w jednym wierszu**. Akapitu nie łamiemy w pliku, bez względu na jego długość; wiersze rozdziela wyłącznie pusta linia. Powód jest praktyczny – porównanie wersji pokazuje wtedy, który akapit się zmienił, zamiast lawiny przesuniętych wierszy po dopisaniu jednego słowa. Ma to znaczenie przy pracy z promotorem i w historii repozytorium, a edytory i tak zawijają długie wiersze na ekranie. Zasada nie dotyczy wnętrza listingów, tabel i wzorów ani pozycji list wypunktowanych.

Twarda spacja zapisywana tyldą nie pozwala złamać wiersza w danym miejscu. Używaj jej tam, gdzie przeniesienie wygląda źle:

```
rysunek~\ref{rys:mapa} s.~15-21 prof.~Kowalski nr~3
```

1.2.3 Polskie znaki i typografia

Piszesz normalnie: zażółć gęślą jaźń. Plik musi być zapisany w UTF-8; Overleaf i Positron robią to domyślnie.

Cudzysłówów nie wpisuj ręcznie. Polecenie `\enquote` dobiera je do języka i poprawnie zagnieżdża, dokładnie tak, jak przewiduje Instrukcja (Instrukcja ISI).

Tabela 2. Zapis typograficzny

Chcesz	Piszesz	Efekt
cudzysłów polski	<code>\enquote{cytat}</code>	„cytat”
cytat w cytacie	<code>\enquote{a \enquote{b} c}</code>	„a «b» c”
półpauza	--	—
łącznik	-	-

W seminarium obowiązuje jedna zasada typograficzna wykraczająca poza wymogi Instytutu: **nie stosujemy pauzy**, czyli trzech łączników. Zdanie wtrącone wydzielamy półpauzą z odstępem po obu stronach – tak jak w tym zdaniu. Łącznik pozostaje łącznikiem w wyrazach złożonych i w zakresach stron.

Kilka znaków ma w LaTeX-u znaczenie specjalne i żeby pojawiły się w tekście, trzeba je poprzedzić ukośnikiem: procent, dolar, ampersand, podkreślenie, kratka i nawiasy klamrowe. Najczęściej potykasz się o podkreślenie w nazwach funkcji – dlatego do nazw służą polecenia opisane w podrozdziale 1.2.7, które robią to za Ciebie.

1.2.4 Rozdziały i podrozdziały

```
\chapter{Podstawy metodyczne}      % 1
\section{Aparat formalny}           % 1.1
\subsection{Normalizacja}           % 1.1.1
```

Numeracja jest automatyczna. Nie wpisuj numerów ręcznie – po wstawieniu rozdziału w środku przenieś numerowanie z godzinę, a nie sekundę. Elementy nienumerowane, obecne mimo to w spisie treści, mają własne polecenia: `\wprowadzenie`, `\podsumowanie`, `\wykazzrodeł`, `\aneksy`.

1.2.5 Etykiety i odsyłacze

To jest funkcja, dla której warto uczyć się LaTeX-a. Nadajesz obiektowi etykietę i odwołujesz się do niej po nazwie; numer wstawia się sam i zawsze jest poprawny.

```
\section{Aparat formalny}\label{sec:aparat}

Metode omowiono w podrozdziale~\ref{sec:aparat} na stronie~\pageref{sec:aparat}.
```

Odsyłacz do nieistniejącej etykiety daje w gotowym pliku dwa znaki zapytania i ostrzeżenie `Reference ... undefined`. Zawsze sprawdzaj je przed oddaniem.

Tabela 3. Konwencja przedrostków w etykietach

Przedrostek	Obiekt
sec:	rozdział, podrozdział
tab:	tabela
rys:	rysunek
wyk:	wykres
lst:	listing
eq:	wzór

1.2.6 Cytowania

```
\parencite{cisek2002filozoficzne}           % (Cisek 2002)
\parencite[s.~15-21]{kowalski2010a}         % (Kowalski 2010a, s. 15-21)
\parencite{klucz1,klucz2}                   % (A 2019; B 2020)
\textcite{wozniak1997kognitywizm} pokazuje, ze... % Wozniak (1997) pokazuje, ze...
```

Postać cytowania – inicjał przy zbieżnych nazwiskach, skrót „i in.” przy więcej niż trzech autorach, sufiksy rocznika przy tym samym roku – dobiera styl automatycznie. Twoim zadaniem jest poprawny wpis w pliku bibliograficznym, opisany w rozdziale 2.

1.2.7 Polecenia własne wzoru

Nie sięgaj po polecenia formatujące bezpośrednio. Wzór ma polecenia, które mówią, **czym** coś jest, i dobierają wygląd zgodnie z wymogami Instytutu.

Tabela 4. Polecenia semantyczne wzoru

Polecenie	Zastosowanie	Wygląd
<code>\fun{przygotuj_dane}</code>	nazwa funkcji	<code>przygotuj_dane()</code>
<code>\argument{skala}</code>	nazwa argumentu	<code>skala</code>
<code>\pkg{ggplot2}</code>	nazwa pakietu	<code>ggplot2</code>
<code>\plik{R/algorytm.R}</code>	plik lub katalog	<code>R/algorytm.R</code>
<code>\kod{x ← 1}</code>	krótki kod w zdaniu	<code>x ← 1</code>
<code>\termin{fuzyfikacja}</code>	termin, wyraz obcy, tytuł	<i>fuzyfikacja</i>
<code>\wyznien{wazne}</code>	wyróżnienie treściowe	ważne
<code>\osoba{Nowak}{Jan}</code>	osoba	Jan Nowak plus wpis do indeksu

Rozróżnienie kursywy i wytłuszczenia nie jest kwestią gustu: Instrukcja przewiduje kursywę dla wyrazów obcych i tytułów, a wytłuszczenie dla wyróżnień w tekście (Instrukcja ISI).

Polecenie `\osoba` stosuj **od pierwszego dnia**. Indeks nazwisk powstaje z tych poleceń, a uzupełnianie go na końcu to kilka godzin pracy.

1.3 Poziom drugi: tabele, rysunki, wzory

1.3.1 Listy i cytaty

Listy stosuj tam, gdzie kolejność albo równorzędność elementów jest istotna. Nie zastępuj nimi wywodu – praca złożona z wypunktowań czyta się jak prezentacja i tak jest oceniana.

Fragment dłuższy niż dwa lub trzy zdania wyodrębniasz graficznie, czego wymaga Instrukcja:

```
\begin{quote}
Kryterium, w którym oceny wariantów są do siebie zbliżone, niesie mało
informacji różnicującej i powinno otrzymać wagę niższą.
\end{quote}
```

1.3.2 Tabele

Tabela w pracy ma tytuł nad sobą i źródło pod sobą. Robi to środowisko `tabelaepi`, które przyjmuje tytuł i etykietę.

```
\begin{tabelaepi}{Moduły pakietu}{tab:moduły}
\begin{tabular}{@{}lll@{}}
\toprule
Plik & Odpowiedzialność & Funkcje eksportowane \\
\midrule
\plik{algorytm.R} & obliczenia & \fun{oblicz_ranking} \\
\bottomrule
\end{tabular}
\zrodło{oprac. własne}
\end{tabelaepi}
```

Wnętrze `tabular` czyta się tak. Deklaracja kolumn podaje ich wyrównanie: `l` do lewej, `r` do prawej, `c` do środka, `p{3cm}` kolumna o stałej szerokości z łamaniem wiersza. Zapis `@{}` na brzegach usuwa wcięcie. Komórki rozdziela ampersand, a wiersz kończą dwa ukośniki. Linie poziome dają `\toprule`, `\midrule` i `\bottomrule` – i tylko one. Tabela naukowa nie ma linii pionowych ani kratownicy.

Tabela szersza niż strona: użyj `tabularx` z kolumną `X`, która sama dobierze szerokość. Tabela dłuższa niż strona przenosi się do aneksu, tak stanowi Instrukcja.

1.3.3 Rysunki

```
\begin{rysunekepi}{Mapa decyzyjna wariantów}{rys:mapa}
```

```
\includegraphics[width=0.78\textwidth]{przyklad-wykres.png}
\zrodlo{oprac. własne}
\end{rysunekpi}
```

Pliki graficzne trzymaj w katalogu `rysunki/` – klasa szuka ich tam automatycznie, więc podajesz samą nazwę pliku. Szerokość podawaj względem szerokości tekstu, nie w centymetrach. Wykresy z R zapisuj w wysokiej rozdzielczości:

```
png("rysunki/wykres.png", width = 1600, height = 1000, res = 220)
plot(x, y)
dev.off()
```

Wykres liczbowy wstawiaj przez środowisko `wykresep`; ma osobną numerację, tak jak wymaga Instrukcja.

Tabele i rysunki są *plywakami*: LaTeX umieszcza je tam, gdzie wychodzi najlepiej, niekoniecznie w miejscu wpisania. To nie jest usterka. Nie walcz z tym w trakcie pisania – zajmij się tym przy redakcji, a w tekście zawsze odsyłaj przez etykietę, nigdy przez zwrot „poniższa tabela”.

1.3.4 Wzory

Wzory składa się w *trybie matematycznym*, w którym symbole ustawiane są inaczej niż zwykły tekst: kursywą i z własnymi odstępami. Wzór w linii otacza się znakami dolara, które ten tryb włączają i wyłączają. Wzór wyróżniony i numerowany zapisuje się tak:

```
\begin{equation}\label{eq:cc}
CC_i = \frac{d_i^-}{d_i^+ + d_i^-}.
\end{equation}
```

We wzorze `\eqref{eq:cc}` symbol `$d_i^+$` oznacza...

Daje to wynik:

$$CC_i = \frac{d_i^-}{d_i^+ + d_i^-}. \quad (1.1)$$

Wzór bez numeru zapisuje się w nawiasach kwadratowych poprzedzonych ukośnikiem. Kilka wzorów wyrównanych do znaku równości składa środowisko `align`.

Symbole zostawiaj takie jak w artykule źródłowym – czytelnik ma moc porównać. I pamiętaj o zasadzie z podrozdziału 4.2: po każdym wzorze następuje akapit objaśniający, co ten wzór wyraża i po co jest w pracy.

Tabela 5. Zapis matematyczny

Chcesz	Piszesz
ułamek	<code>\frac{a}{b}</code>
indeks dolny i górny	<code>x_i, x^2, x_{ij}^{+}</code>
suma, iloczyn	<code>\sum_{i=1}^n, \prod_{j=1}^m</code>
pierwiastek	<code>\sqrt{x}</code>
nawiasy skalujące się	<code>\left(... \right)</code>
litery greckie	<code>\alpha, \lambda, \Phi</code>
wartość bezwzględna	<code>\lvert x \rvert</code>
tekst wewnątrz wzoru	<code>\text{dla } x > 0</code>
macierz	<code>\begin{pmatrix} a & b \\ c & d \end{pmatrix}</code>

1.3.5 Listingi kodu

Listing to fragment kodu wydzielony z tekstu i złożony pismem maszynowym, z własnym numerem i podpisem, tak samo jak tabela czy rysunek. Standardy mówią o *fragmentach kodu* (Standardy EPI); nazwa „listing” jest utrwaloną nazwą tego elementu w pracach informatycznych i tak też podpisuje go wzór, numerując listingi ciągle przez całą pracę.

```
\begin{lstlisting}[caption={Wyznaczanie wag},label={lst:wagi}]
oblicz_wagi <- function(macierz) {
  stopifnot(is.matrix(macierz))
  colSums(macierz) / sum(macierz)
}
\end{lstlisting}
```

Kolorowanie składni, numerację wierszy i ramkę ustawia klasa. Kod z pliku wstawiś poleceniem `\listingR`, które przyjmuje podpis, etykietę i ścieżkę – wtedy listing zawsze odpowiada aktualnej wersji kodu.

Wnętrze listingu musi być zapisane wyłącznie znakami ASCII. Komentarze w kodzie pisz bez polskich znaków diakrytycznych. Powód jest podwójny. Tablica znaków pakietu składającego listingi obejmuje tylko ASCII, więc polskie litery nie tyle znikają, co **wędrują na początek wyrazu**: zapisane w kodzie słowo **ścieżka** złoży się jako **ścieka**. Usterka jest cicha, bo kompilacja kończy się bez błędu. Sprawdzanie pakietu R zgłasza z kolei znaki spoza ASCII w kodzie źródłowym jako problem przenośności, więc to samo ograniczenie obowiązuje w repozytorium. Podpis listingu jest zwykłym tekstem pracy i polskich znaków używać może; ograniczenie dotyczy wyłącznie wnętrza. Kontrola służy skrypt `tools/sprawdz-listingi.R`, ale wymaga on R, więc działa tylko **lokalnie**. Na **Overleaf** objaw widać wprost w złożonym dokumencie: litery w listingu są poprzerastawiane.

Zostaje pytanie, co zrobić, gdy trzeba pokazać tekst polski złożony pismem maszynowym: komunikat błędu, wynik działania programu, fragment danych. Do tego służy

osobne środowisko wydruk, które **polskie znaki składa poprawnie**, bo korzysta z innego pakietu:

```
\begin{wydruk}
Błąd: kolumna "ocena jakosci" zawiera wartosci spoza zakresu.
\end{wydruk}
```

Cena jest taka, że nie ma tam kolorowania składni ani numeracji wierszy. Podział jest więc prosty: **kod idzie do listingu, wyjście programu do wydruku**. Kodowi ograniczenie i tak nie przeszkadza, bo sprawdzanie pakietu R wymaga ASCII niezależnie od składu.

1.4 Poziom trzeci: przy redakcji

1.4.1 Praca w wielu plikach

Rozdziały są osobnymi plikami włączanymi poleceniem `\input`, podawanym bez rozszerzenia. Dzięki temu nie przewijasz tysiąca wierszy i łatwiej znaleźć miejsce błędu. Żeby przy pisaniu jednego rozdziału kompilować tylko jego, zakomentuj pozostałe polecenia włączenia w pliku głównym. Przed oddaniem odkomentuj wszystkie – inaczej numeracja, spis treści i bibliografia będą niepełne.

Znak procentu wyłącza resztę wiersza. Komentarze służą do notatek dla siebie i znikają z gotowego pliku. Przed oddaniem przejrzyj je i usuń te, które są notatkami roboczymi.

1.4.2 Kontrola typografii

Klasa blokuje *wdowy i sieroty*, czyli pojedyncze wiersze akapitu odcięte na końcu albo na początku strony, ale zdarza się, że LaTeX nie umie złamać zbyt długiego słowa i wypuszcza je poza margines. W dzienniku kompilacji zobaczysz wtedy komunikat `Overfull \hbox`. Naprawiaj w tej kolejności: przeformułuj zdanie, co najczęściej jest najlepszym rozwiązaniem; podpowiedz miejsce podziału zapisem `wielo\kryterialny`; przy długim adresie sieciowym użyj polecenia `\url`, które pozwala łamać w sensownych miejscach. Liczba nadmiarowych pudełek w gotowej pracy powinna być zerowa albo bliska zeru.

1.4.3 Sekwencja kompilacji

Pełne złożenie pracy wymaga kilku przebiegów, bo spis treści, odsyłacze, bibliografia i indeks potrzebują danych z przebiegu poprzedniego. Kolejność to LuaLaTeX, potem `biber`, który składa bibliografię z pliku źródeł, potem `makeindex`, który porządkuje indeks nazwisk, a na koniec dwa razy LuaLaTeX.

Na Overleaf. Sekwencja uruchamia się sama po naciśnięciu przycisku kompilacji i nie musisz o niej pamiętać.

Lokalnie. Całą sekwencję uruchamia za Ciebie `latexmk`, wywoływany z katalogu pracy:

```
latexmk -lualatex main.tex
```

Skutek praktyczny jest w obu ścieżkach ten sam: **po dodaniu nowego cytowania albo etykiety pierwsza kompilacja może pokazać znaki zapytania** – po drugiej znikną.

Przy okazji powstaje kilkanaście *plików pomocniczych*, o rozszerzeniach `.aux`, `.toc` czy `.bbl`. To w nich LaTeX przechowuje między przebiegami numery rozdziałów, strony odsyłaczy i złożoną bibliografię. Przydaje się to do momentu, w którym kompilacja zaczyna zachowywać się dziwnie, bo uszkodzony plik pomocniczy potrafi utrzymywać błąd, którego w źródle już nie ma. Czyszczenie wygląda wtedy różnie w obu ścieżkach: **lokalnie** usuwa je polecenie `latexmk -c`, **na Overleaf** służy do tego *Recompile from scratch* z rozwijanego menu obok przycisku kompilacji.

1.5 Diagnostyka

1.5.1 Jak czytać błąd

Komunikat zaczyna się od wykrzyknika, a numer wiersza stoi w linii rozpoczynającej się od litery `ℓ` z kropką. Trzy rzeczy warto wiedzieć. Wiersz wskazany w błędzie bywa o jeden dalej niż przyczyna, bo brakujący nawias zamykający zgłasza się dopiero tam, gdzie coś przestało się zgadzać. Pierwszy błąd jest ważny, a reszta zwykle z niego wynika, więc napraw pierwszy i skompiluj ponownie, zamiast czytać wszystkie. Na Overleaf przełącz widok na pełny dziennik, gdy podsumowanie nie wystarcza.

1.5.2 Katalog błędów

1.5.3 Gdy nic nie pomaga

Wykonaj pełną kompilację od zera po usunięciu plików pomocniczych. Następnie zakomentuj połowę treści i skompiluj, żeby zawęzić, w której połowie jest problem; powtórz kilka razy, aż zostanie kilka wierszy. Na koniec sprawdź, czy problem występuje też w dokumencie pokazowym dołączonym do wzoru – jeżeli tak, przyczyna leży w środowisku, a nie w Twoim tekście.

Tabela 6. Najczęstsze błędy kompilacji

Komunikat albo objaw	Przyczyna	Naprawa
Undefined control sequence	literówka w poleceniu	sprawdź pisownię w tabeli 4
Missing \$ inserted	znak matematyczny poza trybem matematycznym	otocz znakami dolara albo poprzedź ukośnikiem
Missing \begin{document}	tekst wpisany w preambule	przenieś treść za początek dokumentu
File ... not found	zła nazwa albo ścieżka grafiki	grafiki trzymaj w rysunki/
Environment ... undefined	literówka w nazwie środowiska	porównaj początek i koniec
\begin ended by \end	niedomknięte środowiska	sprawdź parowanie
Extra alignment tab	za dużo ampersandów w wierszu	policz kolumny w deklaracji
Misplaced alignment tab	ampersand poza tabelą	poprzedź go ukośnikiem
Citation ... undefined	klucz nieobecny w pliku bibliograficznym	sprawdź klucz, skompiluj dwa razy
Reference ... undefined	odsyłacz do nieistniejącej etykiety	sprawdź etykietę, skompiluj dwa razy
Empty bibliography	brak cytowań w tekście	dodaj cytowanie
Overfull \hbox	tekst wychodzi poza margines	przeformułuj zdanie
Underfull \hbox	zbyt rozstrzelony wiersz	zwykle można zignorować
There's no line here to end	podwójny ukośnik bez wiersza	usuń go, do akapitu wystarczy pusta linia
Paragraph ended before ...	brakujący nawias klamrowy	sprawdź parowanie
Package inputenc Error	plik w innym kodowaniu niż UTF-8	zapisz ponownie w UTF-8
przestawione polskie litery w listingu	znaki spoza ASCII w kodzie	usuń diakrytykę
kod bez właściwego kroju pisma	dokument złożony pdfLaTeX-em	ustaw kompilator na LuaLaTeX
brak indeksu nazwisk	brak polecenia \osoba	dodaj polecenia i skompiluj
dokument nie odświeża się mimo zmian	uszkodzone pliki pomocnicze	pełne złożenie od zera; lokalnie <code>latexmk -c</code>

2 Bibliografia: plik BibTeX i Zotero

Bibliografia w pracy powstaje automatycznie z pliku `bibliografia/literatura.bib`. Nie składasz jej ręcznie i nie sortujesz – robi to styl odwzorowujący opisy z Instrukcji (Instrukcja ISI). Twoje zadanie sprowadza się do jednego: poprawnych wpisów w pliku. Opis wychodzi wtedy zgodny z wymogami sam.

Zasada, której pilnuje recenzent: bibliografia zawiera wyłącznie pozycje faktycznie wykorzystane, do których odsyłają cytowania w tekście. Pozycje przeczytane, ale niewykorzystane, do bibliografii nie trafiają.

Prowadź plik bibliograficzny od pierwszego dnia. Odtwarzanie bibliografii po napisaniu pracy zajmuje kilka dni i zawsze kończy się brakującym numerem strony.

2.1 Anatomia wpisu

```
@book{cisek2002filozoficzne,  
  author    = {Cisek, Sabina},  
  year      = {2002},  
  title     = {Filozoficzne aspekty informacji naukowej},  
  location  = {Krakow},  
  publisher = {Wydaw. UJ},  
  langid    = {polish},  
}
```

Powyższy fragment to jeden *wpis*, czyli komplet danych o jednym źródle. Otwiera go *typ wpisu*, tutaj `@book`, i to on przesądza o postaci gotowego opisu bibliograficznego. Zaraz za nawiasem klamrowym stoi *klucz cytowania*, tutaj `cisek2002filozoficzne`: nazwa, którą nadajesz sam i którą odwołujesz się do tego wpisu, pisząc pracę. Dalej wymienione są *pola* w postaci `nazwa = {wartosc}`, po jednym w wierszu; wartość zawsze stoi w nawiasach klamrowych.

Cytowanie `\parencite{cisek2002filozoficzne}` daje w tekście „(Cisek 2002)”. Klucz jest jedynym łącznikiem między poleceniem w rozdziale a wpisem w pliku i w gotowym dokumencie nigdzie się nie pojawia.

2.1.1 Klucze

Klucz musi być niepowtarzalny w obrębie pliku, a złożyć go wolno wyłącznie z liter, cyfr, łącznika i podkreślenia. Poza tym możesz go zbudować dowolnie, więc przy stu pozycjach opłaca się jedna konwencja zamiast stu pomysłów.

W seminarium klucz składa się z nazwiska, roku i jednego słowa z tytułu, po którym rozpoznasz pozycję na pierwszy rzut oka, zawsze bez znaków diakrytycznych i bez spacji. Przy jednym autorze bierzesz jego nazwisko, przy dwóch oba, drugie od wielkiej litery, przy trzech i więcej samo nazwisko pierwszego:

Tabela 7. Budowa klucza cytowania

Pozycja	Klucz
Peña, Jose M. (2024). <i>Bounds and sensitivity analysis...MNAR confounding</i>	pena2024mnar
Li, Ang; Pearl, Judea (2024). <i>Probabilities of causation...</i>	liPearl2024probabilities
Oosterhuis, Harrie i in. (2024). <i>Reliable confidence intervals...</i>	oosterhuis2024reliable

Klucz raz nadany zostaje na zawsze. Zmiana klucza po napisaniu połowy pracy oznacza przeszukanie wszystkich rozdziałów, a każde przeoczone miejsce daje w złożonym dokumencie znak zapytania zamiast cytowania.

2.1.2 Pola nazwisk

Autorów rozdziela słowo **and**, **nigdy przecinek**. Zapis **Nazwisko**, **Imię** jest jednoznaczny i jego się trzymaj. Przy nazwisku złożonym albo nazwie instytucji użyj podwójnych nawiasów, żeby całość została potraktowana jako jedna jednostka:

```
author = {Nowak, Jan and Zielinski, Kazimierz}
author = {{Biblioteka Jagiellonska}}
author = {van Dijk, Teun A.}
```

Imiona podawaj w pełnym brzmieniu – Instrukcja wymaga pełnych imion w opisie bibliograficznym. Inicjały w cytowaniu styl doda sam, gdy będą potrzebne do odróżnienia dwóch autorów o tym samym nazwisku.

Polskie znaki wpisuj wprost, a plik zapisz w UTF-8. LaTeX zmienia wielkość liter w tytułach w części stylów, więc skrót albo nazwę własną otocz dodatkowymi nawiasami kłamrowymi: `title = {Zastosowanie metody {TOPSIS}}`.

2.2 Typy wpisów

Poniższe wpisy odpowiadają wprost przykładom opisu z Instrukcji. Komplet z komentarzami znajdziesz w pliku bibliograficznym dołączonym do wzoru – zacznij od skopiowania właściwego wzorca i podmiany wartości.

2.2.1 Książka i praca zbiorowa

Książka wielu autorów różni się od pojedynczej tylko listą w polu **author**. Praca zbiorowa ma redaktora zamiast autora, a styl sam dopisze skrót „red.”:

```
@collection{zielinski1998swiat,  
  editor    = {Zielinski, Jan},  
  year      = {1998},  
  title     = {Swiat komputerow},  
  location  = {Wroclaw},  
  publisher = {Globus},  
  langid    = {polish},  
}
```

Daje to opis „Zieliński, Jan red. (1998). *Świat komputerów*. Wrocław: Globus.”, a w tekście cytowanie „(Zieliński red. 1998)”.

2.2.2 Rozdział w pracy zbiorowej

```
@incollection{nowak2004srodowisko,  
  author    = {Nowak, Jan},  
  year      = {2004},  
  title     = {Srodowisko informacyjne},  
  editor    = {Makowski, Eugeniusz},  
  booktitle = {Człowiek współczesny},  
  location  = {Warszawa},  
  publisher = {Wydaw. Atrakcja},  
  pages     = {11-28},  
  langid    = {polish},  
}
```

2.2.3 Artykuł

W pozycjach polskich podajesz **numer** w polu **number**, w obcojęzycznych zwyczajowo **tom** w polu **volume**. Pole **langid** przełącza skróty na oryginalne, dzięki czemu artykuł angielski dostaje oznaczenia „vol.” i „pp.” zamiast „nr” i „s.”:

```
@article{hjørland1998theory,  
  author    = {Hjørland, Birger},  
  year      = {1998},  
  title     = {Theory and metatheory of information science},  
  journaltitle = {Journal of Documentation},  
  volume    = {5},  
  pages     = {606-621},
```

```
  langid      = {english},  
}
```

W artykule prasowym datę dzienną wpisz w pole **issue**; stanie wtedy między numerem a stronami, zgodnie z przykładem z Instrukcji.

2.2.4 Hasło w wydawnictwie informacyjnym

Hasło z własnym autorem opisuje się jak rozdział w pracy zbiorowej. Hasło bez autora wymaga podania autora dzieła nadrzędnego w polu **bookauthor**:

```
@incollection{apoteoza1996,  
  title      = {Apoteoza},  
  bookauthor = {Kopalinski, Władysław},  
  year       = {1996},  
  booktitle  = {Słownik wyrazów obcych},  
  location   = {Warszawa},  
  publisher  = {Wiedza Powszechna},  
  pages      = {14},  
  langid     = {polish},  
}
```

2.2.5 Dokumenty sieciowe

Do wpisu drukowanego dodajesz pola **doi**, **url** i **urldate**. Datę odczytu zapisuj w formacie rok-miesiąc-dzień; styl przestawi ją na polską postać z kropkami. Strony internetowe i wpisy w blogach opisuje typ **@online**, a pole **keywords** o wartości **netografia** pozwala złożyć te pozycje w odrębnym wykazie. *Netografia* to wydzielona część bibliografii obejmująca wyłącznie źródła sieciowe; podział stosujesz tylko wtedy, gdy uzgodnisz go z promotorem.

2.2.6 Pozycje szczególne

Dokument na nośniku materialnym opisuje typ nośnika w polu **howpublished**. Norma wymaga pól **entrysubtype** o wartości **norma** oraz **shorthand**, bo rok wchodzi w skład jej oznaczenia i nie powtarza się go w nawiasie (PN-ISO 690: 2012). Pozycja bez autora potrzebuje pola **shorttitle** z dwoma pierwszymi słowami tytułu, żeby cytowanie miało postać wymaganą przez Instrukcję.

Tabela 8. Pola wpisu bibliograficznego

Pole	Kiedy	Uwaga
author lub editor	zawsze, gdy są	zapis nazwisko, imię; rozdzielone słowem and
year	zawsze	sam rok; brak roku wolno pominąć
title	zawsze	
shorttitle	pozycje bez autora	dwa pierwsze słowa tytułu
journaltitle	artykuł	pełny tytuł, bez skrótów
booktitle	rozdział, hasło	tytuł dzieła nadrzędnego
bookauthor	hasło bez autora	autor dzieła nadrzędnego
number	pozycje polskie	daje oznaczenie „nr 2”
volume	pozycje obcojęzyczne	daje oznaczenie „vol. 5”
issue	gazeta	data dzienna
pages	artykuł, rozdział	zakres zapisany łącznikiem
location	wydawnictwo zwarte	miasto
publisher	wydawnictwo zwarte	
doi	gdy jest	sam identyfikator, bez adresu
url	dokument sieciowy	
urldate	zawsze przy adresie	format rok-miesiąc-dzień
langid	zawsze	polish albo english
keywords	opcjonalnie	wartość netografia
howpublished	nośnik materialny	na przykład płyta DVD
entrysubtype	norma	wartość norma
shorthand	norma	oznaczenie użyte w cytowaniu

2.3 Zestawienie pól

Pole `langid` decyduje o skrótach w opisie, a jego brak jest traktowany jak pozycja polska. **Wypełniaj je zawsze** – inaczej artykuł angielski dostanie polskie oznaczenia tomu i stron.

2.4 Zotero

Ręczne pisanie stu wpisów jest wykonalne, ale nierozsądne. Zotero zbiera metadane z przeglądarki i eksportuje je do pliku bibliograficznego.

2.4.1 Konfiguracja

Zainstaluj Zotero wraz z wtyczką do przeglądarki, a następnie dodatek Better BibTeX. Dodatek jest niezbędny: daje stabilne klucze cytowania, eksport w formacie biblatex i automatyczne odświeżanie pliku. Utwórz kolekcję dla pracy, a w niej podkolekcje na metodę, dziedzinę i narzędzia.

W ustawieniach dodatku ustaw wzorzec klucza odpowiadający konwencji seminaryjnej i **włącz utrwalanie kluczy** – w dodatku polecenie nazywa się *pin*. Bez tego klucz zmieni się po każdej poprawce metadanych, a cytowania w pracy przestaną działać.

Eksport ustawia się raz: menu podręczne kolekcji, polecenie eksportu, format *Better BibLaTeX*, opcja utrzymywania pliku w aktualności. Jako cel wskaż plik bibliograficzny w projekcie pracy. Wybieraj format biblatex, a nie starszy BibTeX. Plik ma w obu przypadkach rozszerzenie `.bib` i wygląda podobnie, ale różni się zestawem pól, a styl instytutowy korzysta z pól, których starszy format nie zna.

2.4.2 Odpowiedniki typów

Tabela 9. Typy dokumentów w Zotero i we wpisie bibliograficznym

Typ w Zotero	Typ wpisu
Book	@book
Book Section	@incollection
Journal Article	@article
Magazine, Newspaper Article	@article
Conference Paper	@inproceedings
Thesis	@thesis
Report	@report
Web Page, Blog Post	@online
Computer Program	@software
Dictionary, Encyclopedia Entry	@incollection

2.4.3 Poprawki po imporcie

Metadane z baz bibliograficznych bywają niekompletne. Po każdym imporcie sprawdź imiona, bo bazy notorycznie zostawiają inicjały, a Instrukcja wymaga pełnych imion. Uzupełnij pole języka wartością `polish` albo `english`, bo stąd bierze się `langid`, i po eksporcie zajrzyj do pliku, czy pole rzeczywiście przeszło. Rozwiń skrócone tytuły czasopism. Sprawdź typ dokumentu, bo rozdział zaimportowany jako artykuł da zły opis. Uzupełnij miejsce wydania, które Zotero często gubi przy książkach. Usuń przedrostek adresowy z identyfikatora cyfrowego. Zapisz tytuł tak, jak w oryginale publikacji, bo bazy anglojęzyczne stosują kapitaliki wyrazowe.

Zasada praktyczna: **nigdy nie ufaj pierwszemu importowi**. Sprawdzenie wpisu zajmuje minutę, a poprawianie bibliografii tydzień przed obroną – znacznie więcej.

2.5 Cytowania w tekście

Tabela 10. Postacie cytowania

Sytuacja	Polecenie	Efekt
zwykle	<code>\parencite{klucz}</code>	(Cisek 2002)
ze stroną	<code>\parencite[s.~15-21]{klucz}</code>	(Kowalski 2010a, s. 15–21)
kilka pozycji	<code>\parencite{k1,k2}</code>	(Cisek 2002; Nowak 2006)
w zdaniu	<code>\textcite{klucz}</code>	Woźniak (1997) pokazuje...
z przedrostkiem	<code>\parencite[zob.][s.~9]{klucz}</code>	(zob. Nowak 2006, s. 9)

Inicjał przy zbieżnych nazwiskach, skrót „i in.” przy więcej niż trzech autorach oraz *sufiks rocznika* styl dobiera automatycznie. Sufiks to litera dopisywana do roku, gdy ten sam autor ma w bibliografii dwie pozycje z tego samego roku: „(Kowalski 2010a)” i „(Kowalski 2010b)”. Nie wpisuj tych elementów ręcznie w pliku bibliograficznym.

Cytat dosłowny z tłumaczeniem własnym zapisujesz razem z oznaczeniem tłumacza i oryginałem w przypisie dolnym:

```
\enquote{Przetłumaczony fragment} [tłum. własne -- JK]
\parencite[s.~44]{hjørland1998theory}\footnote{\enquote{Original wording}.}
```

2.6 Cytowanie oprogramowania i danych

Pakiety R i zbiory danych trafiają do **wykazu źródeł**, a nie do bibliografii; rozróżnienie jest w podrozdziale 4.6.1. Jeżeli jednak powołujesz się na pakiet w tekście jak na publikację, potrzebujesz wpisu. Dane pobierzesz w konsoli:

```
citation("ggplot2")
toBibtex(citation("ggplot2"))
```

Twój pakiet powinien mieć plik `inst/CITATION`, żeby polecenie `citation()` zwracało poprawny opis. Po nadaniu wydania i uzyskaniu identyfikatora cyfrowego opis własnego pakietu wygląda tak:

```
@software{kowalski2027nazwapakietu,
  author = {Kowalski, Jan},
  year   = {2027},
  title  = {NazwaPakietu: krotki opis przeznaczenia},
  version = {1.0.0},
  doi    = {10.5281/zenodo.00000000},
  url    = {https://github.com/nazwauzytkownika/NazwaPakietu},
  urldate = {2027-05-15},
  langid = {polish},
}
```

Zbiór danych opisujesz jak dokument sieciowy, z podaniem wersji albo daty pobrania oraz licencji w polu `note`.

2.7 Kontrola

Przed oddaniem sprawdź trzy rzeczy. Po pierwsze zgodność w obie strony: każda pozycja bibliografii ma cytowanie w tekście, a każde cytowanie pozycję w bibliografii. Nierozwiązane cytowania widać w dzienniku kompilacji; pozycji bez cytowania styl po prostu nie złoży, co działa na Twoją korzyść. Po drugie obecność pozycji obcojęzycznych, wymaganych zarówno przez Standardy, jak i przez Instrukcję. Po trzecie kompletność opisów – przejrzyj gotową bibliografię w złożonym dokumencie, a nie w pliku źródłowym, bo brakujące miasto, ucięty tytuł czasopisma albo brak daty odczytu widać dopiero w składzie.

Gdy bibliografia jest pusta albo nieaktualna po zmianach w pliku, uruchom pełną kompilację od zera.

2.8 Warianc APA

Jeżeli uzgodnisz z promotorem styl APA zamiast instytutowego, wystarczy dopisać opcję klasy `apa` w poleceniu `\documentclass`. Ten sam plik bibliograficzny obsługuje oba style. Łączniki są polskie, a nazwy miesięcy w datach dostępu pozostają w formie właściwej dla angielskiego zapisu APA.

Wybór stylu uzgadnia się **raz, na początku** – Instrukcja wprost przewiduje ustalenie preferowanego stylu z promotorem. Zmiana w trakcie pisania oznacza przegląd wszystkich wpisów pod kątem pól wymaganych przez nowy styl.

3 Overleaf i współpraca

Rozdział dotyczy ścieżki przeglądarkowej z podrozdziału `efsec:sciezki`; praca lokalna opisana jest w podrozdziale `efsec:lokalnie`.

Overleaf to LaTeX w przeglądarce. Nie trzeba nic instalować, projekt jest dostępny z każdego komputera, a promotor widzi tę samą wersję co Ty. Uniwersytet prowadzi **własną instalację** pod adresem `overleaf.uj.edu.pl` i to na niej pracujemy: projekty zostają w infrastrukturze uczelni, a udostępnianie działa wewnątrz Uniwersytetu. Dla pracy licencjackiej pisanej pod opieką promotora to najmniej kłopotliwe środowisko. Praca lokalna też jest możliwa i opisana jest w podrozdziale 3.4.

3.1 Ścieżka studenta

3.1.1 Konto i projekt

Konta nie zakładasz. Logujesz się przez **centralny punkt logowania** Uniwersytetu, kontem w domenie `@student.uj.edu.pl`. Jeżeli masz konto w serwisie publicznym Overleaf, jest ono niezależne: projektów między jednym a drugim nie widać i nie da się ich przenieść inaczej niż przez pobranie i wgranie archiwum.

Wzór wgrywasz jako **paczkę**. Pobierz plik `wzor-overleaf.zip` ze strony seminarium (<https://marekdejauj.github.io/epi-seminarium-Deja/>), a na Overleaf wybierz zakładanie nowego projektu z przesłanego archiwum i wskaż pobrany plik. Projekt powstaje od razu z gotową strukturą katalogów i niczego nie trzeba układać ręcznie.

Paczka powstaje z repozytorium przy każdej zmianie wzoru, więc pobrana dziś jest zgodna z bieżącą wersją klasy i stylu bibliograficznego. Wariant zapasowy, gdy strona seminarium jest niedostępna: pobierz archiwum całego repozytorium, rozpakuj je i spakuj **zawartość katalogu wzoru** do nowego archiwum.

3.1.2 Ustawienia

Tabela 11. Ustawienia projektu na Overleaf

Ustawienie	Wartość	Dlaczego
Compiler	LuaLaTeX	kod składa się wtedy właściwym kro- jem pisma
Main document	<code>main.tex</code>	inaczej kompiluje się dokument po- kazowy
Spell check	polski	podkreśla literówki w trakcie pisania

Ustawienie kompilatora jest jednorazowe i **łatwe do przeoczenia**. Objaw pominięcia: praca kompiluje się poprawnie, ale kod i nazwy funkcji mają inny krój pisma niż powinny.

3.1.3 Co jest czym w projekcie

W pliku głównym uzupełniasz metadane na górze i nic więcej. Piszesz w plikach rozdziałów. Wpisy bibliograficzne trafiają do pliku w katalogu bibliografii, a grafiki do katalogu rysunków. Dokument pokazowy służy do zaglądania, gdy nie pamiętasz zapisu.

Klasy dokumentu ani plików stylu bibliograficznego **nie ruszasz**. Zmiana czegokolwiek w nich oznacza, że praca przestaje odpowiadać wymogom Instytutu, a odpowiedzialność za to spada na Ciebie, nie na wzór. Jeżeli czegoś nie da się zrobić dostępnymi poleceniami, zgłoś to promotorowi – brakuje wtedy polecenia we wzorze.

3.1.4 Kompilacja i kopie

Pierwsza kompilacja po dodaniu nowego cytowania albo etykiety może pokazać znaki zapytania; po drugiej znikną. Rozwijane menu obok przycisku kompilacji zawiera polecenie pełnego przebiegu od zera – użyj go, gdy dokument przestaje odpowiadać zmianom albo gdy bibliografia nie chce się odświeżyć.

Kompilacja ma ograniczenie czasu. Praca licencjacka mieści się w nim spokojnie, ale gdy zaczniesz się o nie ocierać, zakomentuj w pliku głównym włączenia rozdziałów, nad którymi akurat nie pracujesz. Przed oddaniem odkomentuj wszystkie.

Projekt leży na serwerze uczelni i ma historię zmian, ale **to nie jest kopia zapasowa**. Historia chroni przed Twoim błędem, nie przed awarią serwera ani przed utratą dostępu do konta. Raz w tygodniu pobierz źródła projektu i zapisz je poza tą instalacją; przed każdym większym przemeblowaniem pracy dodatkowo.

3.1.5 Udostępnienie i oddanie

Udostępniij projekt promotorowi **na adres uczelniany**, z prawem edycji. Udostępnianie działa wewnątrz instalacji uczelnianej, więc adres prywatny nie zadziała. Recenzentowi, gdy zajdzie potrzeba, wystarczy łączyć tylko do odczytu.

Do Archiwum Prac trafia plik wynikowy pobrany z projektu. Przed pobraniem wykonaj pełną kompilację od zera, sprawdź brak nierozwiązanych cytowań i odsyłaczy oraz przejdź listę kontrolną z podrozdziału 6.10.

3.2 Plik bibliograficzny a Zotero

Dodatek Better BibTeX odświeża plik bibliograficzny **na Twoim dysku**. Overleaf o tym nie wie, więc plik trzeba do projektu przenieść. Najprostszy sposób to wgranie ręczne:

w drzewie plików wskaż katalog bibliografii, prześlij zaktualizowany plik i potwierdź nadpisanie. Wystarczająco dobre, jeżeli robisz to raz na tydzień. Instalacja może mieć wczytywanie biblioteki Zotero na żądanie; sprawdź w ustawieniach projektu, czy jest dostępne, bo wymaga osobnej konfiguracji po stronie serwera. Przy synchronizacji z repozytorium plik odświeża się razem z resztą projektu.

Niezależnie od sposobu: **po każdej podmianie pliku uruchom pełną kompilację od zera**. Bibliografia budowana jest z pliku pomocniczego, który potrafi zostać na starej wersji.

3.3 Ścieżka promotora

3.3.1 Rozdanie wzoru grupie

Wzór rozdaje się **jako paczkę**, nie jako projekt do skopiowania. Prowadzący podaje jedno łącze do pliku `wzor-overleaf.zip` na stronie seminarium, każda osoba zakłada z niego własny projekt i udostępnia go prowadzącemu z prawem edycji.

Zalety układu: jedno łącze wystarcza dla całej grupy, paczka jest zawsze zgodna z repozytorium, projekty studentów są od początku niezależne, a prowadzący ma dostęp do wszystkich z jednego pulpitu. Wadą jest to, że poprawka we wzorze nie trafia do projektów już założonych – dlatego **wzór zamyka się przed pierwszymi zajęciami**, a późniejsze zmiany rozsyła się jako opis, co podmienić.

3.3.2 Przegląd prac

Instalacja uczelniana daje trzy narzędzia: tryb przeglądu, historię zmian i zwykłą edycję.

Tryb przeglądu to boczny panel z komentarzami. Zaznaczasz fragment tekstu i dopisujesz do niego uwagę; uwaga zostaje przypięta do tego fragmentu, widzą ją obie strony, a po naniesieniu poprawki oznacza się ją jako załatwioną i znika z panelu. To jest podstawowy kanał uwag w seminarium.

Czego **nie ma**: śledzenia zmian. Zdanie poprawione przez prowadzącego wygląda w tekście dokładnie tak samo jak zdanie napisane przez studenta i nie da się go ani wyróżnić, ani odrzucić jednym kliknięciem. Wynika stąd zasada obowiązująca przez cały rok: **prowadzący komentuje, student poprawia**. Jeżeli prowadzący poprawi coś sam, bo tak jest szybciej, zostawia przy tym komentarz mówiący, co zmienił – inaczej zmiana we własnym tekście przejdzie studentowi niezauważona.

Historia zmian pozwala obejrzeć stan projektu z dowolnego dnia i porównać go z bieżącym. Żeby dało się z niej korzystać, **oznaczaj wersję etykietą przy każdym oddaniu fragmentu**. Bez etykiet historia jest ciągłym strumieniem zmian, w którym nie ma punktu odniesienia.

Przy pracy lokalnej w Positronie panelu komentarzy nie ma, więc uwagi zapisuje się jako komentarze LaTeX-a, które **nie pojawiają się w gotowym dokumencie**:

```
%% UWAGA: teza z tego akapitu nie wynika z cytowanego źródła.  
%% UWAGA: brakuje cytowania.  
%% PYTANIE: skąd wartość progu 0,7?
```

Uwaga stoi wtedy dokładnie przy problematycznym zdaniu, student widzi ją przy pisaniu, a gotowy dokument pozostaje czysty. Wyszukanie wszystkich uwag w projekcie to jedno kliknięcie w polu wyszukiwania.

Zasada zamykająca jest w obu trybach ta sama: **uwagę zamyka się dopiero po naniesieniu poprawki**. Brak otwartych komentarzy w panelu i pusty wynik wyszukiwania w źródle oznaczają, że wszystkie uwagi zostały zaadresowane; ten punkt jest na liście kontrolnej przed oddaniem.

3.3.3 Kontrola formalna

Kontrolę formalną prowadź **na złożonym dokumencie, nie w źródle**. Marginesy, interlinia, numeracja stron i układ elementów końcowych widać dopiero w składzie. Zestawienie wymogów wraz ze wskazaniem, co realizuje klasa, a co pozostaje po stronie autora, zawiera rozdział 6.

W źródle warto sprawdzić dwie rzeczy: czy student nie zmieniał klasy dokumentu ani plików stylu bibliograficznego, co ułatwia porównanie z wzorcem, oraz czy w projekcie nie zostały komentarze robocze i uwagi.

Przy kilkunastu pracach przegląd na żądanie nie działa. Sprawdzony układ to stałe okna: oddanie fragmentu do ustalonej daty, przegląd w ciągu tygodnia, omówienie na zajęciach. Fragmenty oddawane w kolejności pisania, a nie w kolejności rozdziałów – kolejność jest w tabeli 17.

3.4 Praca lokalna w Positronie

Skoro pakiet R budujesz w Positronie, możesz w nim też pisać pracę. Kod i tekst są wtedy w jednym środowisku, a wykresy trafiają prosto do katalogu rysunków. Potrzebujesz dystrybucji TeX-a oraz rozszerzenia do LaTeX-a.

```
cd wzor-pracy  
latexmk -lualatex main.tex
```

Zaletami są brak limitu czasu kompilacji, praca bez dostępu do sieci i pełna historia w repozytorium. Wadami – konieczność zainstalowania i utrzymania dystrybucji TeX-a

oraz to, że udostępnienie promotorowi wymaga repozytorium zamiast jednego kliknięcia. Układ mieszany bywa najwygodniejszy: pakiet i wykresy lokalnie, tekst na Overleaf.

Repozytorium pakietu i pracę trzymaj **osobno**. Praca zawiera pliki, które nie mają nic wspólnego z pakietem, a repozytorium pakietu ma być czyste – to ono jest oceniane jako projekt dyplomowy i to jego adres podajesz na stronie tytułowej. Jeżeli prowadzisz pracę w repozytorium, ignoruj artefakty kompilacji.

3.5 Problemy specyficzne dla Overleaf

Tabela 12. Problemy w pracy na Overleaf

Objaw	Przyczyna	Naprawa
kod bez właściwego kroju písma	kompilator ustawiony na pdfLaTeX	zmień kompilator na LuaLaTeX
kompiluje się dokument pokazowy	zły dokument główny	wskaż plik główny pracy
bibliografia nie odświeża się	stary plik pomocniczy	pełna kompilacja od zera
kompilacja przerwana limitem czasu	za dużo naraz przy szkicowaniu	zakomentuj nieużywane włączenia
grafika nie znaleziona	plik poza katalogiem rysunków	przenieś plik, podawaj samą nazwę
polskie znaki jako znaki zastępcze	plik w innym kodowaniu	zapisz ponownie w UTF-8
zmiany współpracownika niewidoczne	stara karta przeglądarki	odśwież stronę
projekt niewidoczny dla promotora	udostępniony na adres prywatny	udostępnij na adres uczelniany
brak katalogów po wgraniu	spakowany katalog zamiast jego zawartości	spakuj zawartość, nie katalog

Katalog błędów kompilacji niezależnych od Overleaf zawiera podrozdział 1.5.

4 Jak napisać pracę

W rozdziale omówiona jest każda część wzoru po kolei. Każdy podrozdział odpowiada jednemu plikowi w katalogu rozdziałów. Kolejność podrozdziałów jest kolejnością w pracy, ale **nie kolejnością pisania** – o tym w podrozdziale 4.7.

Twoja praca opisuje projekt: pakiet języka R implementujący algorytm z artykułu metodycznego. To zmienia sens niektórych części w stosunku do prac czysto teoretycznych. Stanem badań jest tu nie tylko literatura przedmiotu, ale też istniejące oprogramowanie. Materiałem badawczym są dane, na których pokazujesz działanie narzędzia. Wynikiem jest działające, sprawdzone narzędzie oraz wnioski z jego zastosowania.

4.1 Wprowadzenie

Plik `rozdzialy/00-wprowadzenie.tex`, objętość trzy do pięciu stron, bez numeru rozdziału.

Wprowadzenie odpowiada czytelnikowi na cztery pytania w tej kolejności: o czym to jest, dlaczego to problem, co konkretnie zrobiłeś, jak czytać dalej. Recenzent czyta wprowadzenie i podsumowanie uważniej niż resztę – z nich wyrabia sobie zdanie o pracy.

4.1.1 Co musi zawierać

Tło problemu. Zaczynij od problemu, który istnieje niezależnie od Twojego pakietu. Nie od narzędzia, tylko od pytania, na które badacz nauk społecznych dziś nie potrafi odpowiedzieć albo odpowiada z trudem. Dopiero na tym tle pojawia się metoda.

Luka. Wyjaśnij, czego brakuje. W pracy projektowej luka ma dwie warstwy i warto rozróżnić je wprost. *Luka technologiczna* to sytuacja, w której metoda jest opisana w literaturze, ale nie ma jej implementacji; implementacja istnieje, lecz wymaga przygotowania programistycznego; istniejące narzędzie nie obsługuje danych określonego rodzaju. *Luka poznawcza* to z kolei to, czego badacze nie wiedzą, bo nie mają czym policzyć. Praca, która wypełnia wyłącznie pierwszą, jest sprawozdaniem z programowania, a nie pracą naukową.

Luka musi być sprawdzalna. Wymień narzędzia, które przejrzałeś, i napisz, czego w nich nie ma. Zdanie o nieistnieniu implementacji, podane bez wykazu przeszukanych źródeł, jest zarzutem pod Twoim adresem, a nie argumentem.

Cel. Jedno zdanie w formie czynności: celem pracy jest zaprojektowanie, implementacja i weryfikacja pakietu języka R udostępniającego określoną procedurę badaczom nauk społecznych. Cel jest tym, co zrobiłeś, a nie tym, czego się dowiedziałeś.

Pytanie badawcze. Cel mówi, co zbudowałeś; pytanie mówi, czego się dzięki temu dowiadujemy. Pytania zaczynające się od „jak” i „dlaczego” prowadzą do analizy, a „co” i

„ile” do opisu. Sposób dochodzenia do pytania jest w podrozdziale 5.1. Przykłady pytań właściwych dla tego typu pracy:

- Jakie warunki musi spełnić implementacja metody, żeby jej wyniki dały się odtworzyć przez osobę trzecią?
- Które założenia metody okazują się najbardziej wrażliwe na jakość danych spotykanych w badaniach społecznych?
- Jak wybór procedury generowania danych testowych wpływa na to, co wolno wnioskować o poprawności implementacji?

Materiał i metoda. Dwa lub trzy zdania: na jakich danych sprawdzasz narzędzie i jak weryfikujesz poprawność. Szczegóły zostawiasz do rozdziałów drugiego i trzeciego pracy.

Zakres i ograniczenia. Co praca obejmuje, a czego nie. Ograniczenie postawione samodzielnie jest mocniejsze niż ten sam zarzut w recenzji.

Struktura. Jedno zdanie o każdym rozdziale.

4.1.2 Typowe błędy

Tabela 13. Błędy we wprowadzeniu

Błąd	Dlaczego szkodzi
Wprowadzenie zaczyna się od opisu pakietu	czytelnik nie wie jeszcze, po co mu ten pakiet
Cel sformułowany jako przedstawienie zagadnienia	nie da się sprawdzić, czy został osiągnięty
Luka stwierdzona bez wykazu przejranych narzędzi	twierdzenie nieudokumentowane
Historia dziedziny od połowy zeszłego wieku	wprowadzenie ma wprowadzać do problemu, nie do dyscypliny
Pytanie badawcze utożsamione z celem	brakuje warstwy poznawczej pracy

- ☐ Problem postawiony przed narzędziem
- ☐ Luka wskazana i udokumentowana wykazem przejranych rozwiązań
- ☐ Cel w jednym zdaniu, w formie czynności
- ☐ Pytanie badawcze różne od celu
- ☐ Zakres i ograniczenia wypisane
- ☐ Zapowiedź struktury zgodna z faktyczną strukturą pracy
- ☐ Każde twierdzenie o stanie rzeczy ma cytowanie

4.2 Rozdział pierwszy: podstawy metodyczne

Plik `rozdzialy/01-podstawy.tex`, objętość osiem do dwunastu stron.

Odpowiednik sekcji poświęconej podstawom metodycznym w czasopiśmie o oprogramowaniu badawczym. Opisujesz matematykę i logikę, które zaimplementowałeś – jeszcze nie kod. W tym rozdziale masz udowodnić, że rozumiesz metodę, którą zamknąłeś w narzędziu.

4.2.1 Zasada nadrzędna

Po każdym wzorze następuje akapit wyjaśniający, co ten wzór wyraża i po co jest w pracy, językiem zrozumiałym dla badacza bez przygotowania matematycznego. Wzór bez wyjaśnienia jest w tej pracy błędem merytorycznym, a nie oszczędnością miejsca. Twoim odbiorcą jest osoba, która chce metody użyć, a nie ją wyprowadzić.

Sprawdzian: przeczytaj sam akapit, bez wzoru. Czy nadal wiadomo, co się dzieje? Jeżeli nie, akapit jest za słaby.

4.2.2 Co musi zawierać

Osadzenie metody. Jaki problem metoda rozwiązuje, skąd się wzięła, czym różni się od podejść wcześniejszych. Cytowanie artykułu źródłowego oraz dwóch lub trzech opracowań wprowadzających.

Aparat formalny. Wielkości wejściowe wraz z dziedzinami, kolejne kroki algorytmu, wielkości wyjściowe. Numeruj i etykietuj te wzory, do których praca się odwołuje. Objasnij **każdy** symbol, którego używasz – nawet ten, który wydaje Ci się oczywisty.

Założenia i granice stosowalności. Przy jakich warunkach metoda jest poprawna: rozkłady, niezależność obserwacji, kompletność danych, skale pomiarowe. I co się dzieje, gdy założenie jest naruszone. Ta sekcja zasila później kontrakt w specyfikacji oraz testy odporności pakietu – pisząc ją, projektujesz jednocześnie oprogramowanie.

Rozwiązania pokrewne. Przegląd istniejących implementacji w układzie problemowym: co już jest, czego brakuje, jak Twoje narzędzie się do tego ma. Nie lista, tylko wywód. Zakończ jednoznacznym stwierdzeniem, jaki wkład wnosi Twój pakiet.

4.2.3 Skąd brać treść

Z artykułu źródłowego, z sekcji definicyjnej i algorytmicznej. **Nie przepisuj wzorów z opracowań wtórnych ani z zestawień tematycznych** – zapis bywa tam skrócony i niedokładny. Wracaj do publikacji.

Wzoru nie tłumacz na polski w sensie notacji. Symbole zostają takie jak w źródle, żeby czytelnik mógł porównać; tłumaczysz sens, a nie znaki.

Tabela 14. Błędy w rozdziale metodycznym

Błąd	Dlaczego szkodzi
Wzory przepisane bez objaśnienia symboli	recenzent nie odróżni zrozumienia od przepisania
Opis kodu zamiast opisu metody	kod należy do rozdziału drugiego
Pominięcie założeń metody	testy odporności nie mają z czego wynikać
Przegląd narzędzi w formie listy	brak wyводу problemowego, wymóg Standardów
Wzory przepisane z opracowania wtórnego	ryzyko powielenia cudzego błędu

- ☐ Każdy symbol objaśniony w tekście
- ☐ Po każdym wzorze akapit o tym, co wzór wyraża
- ☐ Założenia metody wypisane wprost
- ☐ Granice stosowności opisane
- ☐ Przegląd rozwiązań pokrewnych w układzie problemowym
- ☐ Wkład własny sformułowany jednoznacznie
- ☐ Wzory sprawdzone z publikacją źródłową, nie z zestawieniem

4.3 Rozdział drugi: implementacja i architektura

Plik `rozdzialy/02-implementacja.tex`, objętość dziesięć do czternastu stron.

Odpowiednik sekcji opisującej oprogramowanie. Najważniejszy rozdział inżynierski – tu pokazujesz własny wkład. Standardy wymagają w tym miejscu opisu logiki aplikacji ze schematami oraz opisu implementacji z formatem danych i sposobem realizacji każdego modułu (Standardy EPI).

4.3.1 Co musi zawierać

Struktura projektu. Katalogi, moduły, przepływ danych. Wyjaśnij, co gdzie leży i dlaczego akurat tak. Tabela modułów z odpowiedzialnościami i funkcjami eksportowanymi mówi więcej niż akapit opisu.

Przetwarzanie i walidacja danych. Droga od danych surowych do struktury, na której pracuje algorytm: sprawdzanie warunków wstępnych, obsługa braków i kodów błędnych, skalowanie, agregacja. Wypisz, które naruszenia zatrzymują wykonanie i z jakim komunikatem – to jest kontrakt Twojego narzędzia i najmocniejsza część tego rozdziału.

Silnik obliczeniowy. Jak wzory z rozdziału pierwszego przełożyły się na kod. Pokaż fragment, nie całość; całość jest w repozytorium. Wyjaśnij decyzje, które nie wynikają wprost ze wzoru: sposób radzenia sobie z dzieleniem przez zero, wybór biblioteki optymalizacyjnej, zamianę pętli na operacje na wektorach.

System klas i metody generyczne. Dlaczego wynik jest obiektem określonej klasy, a nie zwykłą ramką danych, i co dzięki temu potrafią funkcje wypisujące, podsumowujące i rysujące.

Procedura generowania danych testowych. Rozkłady, parametry, zależności między zmiennymi, kontrolowane braki i wartości odstające, ziarno losowości. To jest element metodologii, a nie dodatek techniczny: dzięki znanej strukturze danych wiadomo, jaki wynik narzędzie **powinno** zwrócić, więc odchylenie jest wykrywalne.

Weryfikacja poprawności. Trzy warstwy, opisane osobno. Przypadki analityczne o wyniku policzonym ręcznie: mała macierz, przypadek zdegenerowany, wartości brzegowe. Testy własności: niezmienniczość na skalowanie, monotoniczność, odporność na permutację etykiet. Testy odporności: zerowa wariancja, współliniowość, wartości poza dziedziną, macierz osobliwa – oczekiwanym wynikiem jest kontrolowany błąd z czytelnym komunikatem, a nie poprawny wynik. Podaj rezultat sprawdzenia zgodności na trzech systemach operacyjnych.

Narzędzia zewnętrzne. Dla każdej biblioteki nazwa, źródło, przeznaczenie oraz krótki opis wejścia i wyjścia. Wymóg wprost ze Standardów.

4.3.2 Jak pisać o pracy z agentem

Jeżeli korzystałeś z narzędzia wspomagającego pisanie kodu, opisz to jako element metodyki inżynierskiej, a nie jako wyznanie. Interesujące jest to, **gdzie specyfikacja wychwyciła błąd**: w którym miejscu narzędzie uprościło założenie metody i jak to wykryłeś. To pokazuje zrozumienie metody lepiej niż bezbłędny kod. Zasada podstawowa pozostaje jedna: musisz umieć objaśnić każdą linię kodu, którą oddajesz.

Tabela 15. Błędy w rozdziale inżynierskim

Błąd	Dlaczego szkodzi
Wklejone całe pliki źródłowe	praca nie jest wydrukiem repozytorium
Opis funkcji zamiast opisu architektury	dokumentacja funkcji jest na stronie projektu
Procedura generowania danych zbyt jednym zdaniem	traci sens cała weryfikacja
Brak opisu kontraktu błędów	nie wiadomo, kiedy narzędzie zawodzi świadomie
Testy opisane jako napisano testy jednostkowe	nie wiadomo, co właściwie sprawdzono

- ☐ Schemat lub tabela modułów z odpowiedzialnościami
- ☐ Opisany kontrakt: warunki wstępne, komunikaty błędów
- ☐ Fragmenty kodu, nie całe pliki
- ☐ Procedura generowania danych opisana z parametrami i ziarnem
- ☐ Trzy warstwy weryfikacji opisane osobno
- ☐ Wynik sprawdzenia zgodności na trzech systemach
- ☐ Biblioteki zewnętrzne z nazwą, źródłem, przeznaczeniem, wejściem i wyjściem
- ☐ Listingi zawierają wyłącznie znaki ASCII

4.4 Rozdział trzeci: studium przypadku

Plik `rozdzialy/03-studium-przypadku.tex`, objętość osiem do dziesięciu stron.

Pokazujesz, że narzędzie działa na danych, a nie tylko się kompiluje. Zagadnienie ma należeć do dziedziny wskazanej w tytule pracy – inaczej tytuł przestaje odpowiadać treści, co jest pierwszym kryterium oceny w Standardach.

4.4.1 Co musi zawierać

Charakterystyka danych. Pochodzenie, licencja, liczba obserwacji i zmiennych, skale pomiarowe, braki. Jeżeli używasz danych z własnej procedury, odeślij do rozdziału drugiego i podaj przyjęte parametry. Napisz, jak wybór zbioru wpływa na to, co wolno wywnioskować.

Dane wygenerowane pozwalają sprawdzić poprawność, bo znasz prawdziwą odpowiedź. Dane empiryczne pozwalają pokazać użyteczność, ale prawdziwej odpowiedzi nie znasz. Najmocniejszy rozdział trzeci używa obu i mówi wprost, po co każdego.

Przebieg analizy. Pełna ścieżka od danych surowych do wyniku, w postaci listingu. Czytelnik ma móc ją powtórzyć.

Wyniki. Tabela i rysunek, a po nich interpretacja w tekście. Tabela nie zastępuje interpretacji. Oddziel to, co pokazuje narzędzie, od tego, co twierdzisz Ty: narzędzie zwraca liczby, a wniosek merytoryczny jest Twój i wymaga uzasadnienia.

Porównanie. Zestaw wyniki z rezultatem z artykułu źródłowego albo z metodą odniesienia. Rozbieżność nie jest porażką – jest wynikiem, o ile potrafisz wskazać jej przyczynę. Rozbieżność przemilczana jest błędem.

- ☐ Zbiór opisany: pochodzenie, licencja, rozmiar, skale, braki
- ☐ Uzasadniony wybór danych względem tematu pracy
- ☐ Pełna ścieżka analizy możliwa do powtórzenia
- ☐ Każdy wynik zinterpretowany w tekście
- ☐ Rozdzielone: co pokazuje narzędzie, a co twierdzi autor

Tabela 16. Błędy w studium przypadku

Błąd	Dlaczego szkodzi
Zbiór danych spoza dziedziny z tytułu pracy	treść przestaje odpowiadać tematowi
Tabela wyników bez interpretacji	czytelnik ma wykonać pracę autora
Wniosek mocniejszy niż dane	najczęstszy zarzut w recenzjach
Brak informacji o licencji danych	uniemożliwia odtworzenie analizy
Pominięcie rozbieżności z artykułem źródłowym	podważa wiarygodność weryfikacji

- ☐ Porównanie z wynikiem odniesienia wraz z omówieniem rozbieżności

4.5 Podsumowanie

Plik `rozdzialy/04-podsumowanie.tex`, objętość dwie do trzech stron, bez numeru rozdziału.

Podsumowanie domyka klamrę z Wprowadzeniem. Czytelnik, który przeczyta tylko te dwie części, ma wiedzieć, co zostało zrobione i z jakim skutkiem. Zawiera: realizację celu podaną wprost, odpowiedź na pytanie badawcze wynikającą z rozdziałów drugiego i trzeciego, trzy do pięciu wniosków uporządkowanych od najważniejszego, ograniczenia oraz perspektywy rozwoju aplikacji. Ostatni element jest wymogiem wprost ze Standardów – nie zbywaj go jednym zdaniem.

Nie wprowadzaj tu niczego nowego i nie powołuj się na nowe źródła. Jeżeli w trakcie pisania podsumowania pojawia się nowy argument, jego miejsce jest wcześniej.

- ☐ Powrót do celu z Wprowadzenia
- ☐ Odpowiedź na pytanie badawcze
- ☐ Wnioski uporządkowane od najważniejszego
- ☐ Ograniczenia
- ☐ Perspektywy rozwoju opisane konkretnie
- ☐ Zero nowych źródeł i nowych tez

4.6 Elementy końcowe

4.6.1 Wykaz źródeł

Plik `rozdzialy/05-wykaz-zrodel.tex`. To **nie to samo co bibliografia**. Bibliografia obejmuje opracowania naukowe, do których odsyłają cytowania w tekście. Wykaz źródeł obejmuje narzędzia i dane: pakiety R, zbiory danych, serwisy udostępniające oprogra-

mowanie. Standardy dopuszczają reprezentowanie narzędzi programistycznych adresami serwisów, które je udostępniają i opisują.

4.6.2 Bibliografia

Generowana automatycznie z pliku bibliograficznego. Zasada, której pilnuje recenzent: żadnej pozycji w bibliografii bez cytowania w tekście i odwrotnie. Pozycje przeczytane, ale niewykorzystane, do bibliografii nie trafiają. Muszą się w niej znaleźć pozycje obcojęzyczne – wymóg zarówno Standardów, jak i Instrukcji. Zapis wpisów jest w rozdziale 2.

4.6.3 Spis ilustracji i indeks nazwisk

Powstają automatycznie. Indeks zbiera nazwiska wstawione poleceniem `\osoba` – używaj go od pierwszego dnia pisania, bo uzupełnianie indeksu na końcu to praca na kilka godzin.

4.6.4 Aneksy

Plik `rozdzialy/06-aneksy.tex`. Wzór przewiduje cztery aneksy: metadane oprogramowania, notę wymaganą w serwisie internetowym zgodnie z załącznikiem do Standardów, oświadczenie o wykorzystaniu sztucznej inteligencji oraz wykaz poleceń wydanych narzędziu programistycznemu. Ostatni dotyczy wyłącznie budowy aplikacji i dla każdego polecenia podaje cel, treść, wprowadzone przez Ciebie zmiany i sposób weryfikacji. Aneksy muszą być powiązane z tekstem odsyłaczami.

4.7 W jakiej kolejności pisać

Wprowadzenie pisane jako pierwsze jest prawie zawsze pisane dwa razy.

Trzy rzeczy prowadź **od pierwszego dnia**, a nie na końcu: plik bibliograficzny, polecenia wstawiające nazwiska do indeksu oraz rejestr poleceń wydanych narzędziu programistycznemu. Odtwarzanie każdej z nich po fakcie kosztuje wielokrotnie więcej czasu.

Tabela 17. Kolejność pisania rozdziałów

Etap	Co piszesz	Dlaczego wtedy
1	rozdział 1, aparat formalny	zmusza do zrozumienia metody, zanim powstanie kod
2	rozdział 1, założenia i granice	staje się kontraktem specyfikacji pakietu
3	rozdział 2, generowanie danych	piszesz razem z kodem, gdy pamiętasz parametry
4	rozdział 2, reszta	po zamknięciu implementacji
5	rozdział 3	po pierwszym pełnym przebiegu analizy
6	rozdział 1, rozwiązania pokrewne	dopiero teraz wiesz, z czym się porównujesz
7	Wprowadzenie	wiadomo już, co się zapowiada
8	Podsumowanie	domyka klamrę z gotowym Wprowadzeniem
9	aneksy, wykaz źródeł, redakcja	na końcu

5 Warsztat pisania akademickiego

Rozdział jest o tym, **jak** pisać: skąd wziąć pytanie, jak zbudować argument, jak rozmawiać z literaturą i jak nie stracić panowania nad źródłami. Co ma się znaleźć w której części pracy, jest w rozdziale 4; czego wymaga Instytut – rozdział 6.

5.1 Pytanie analityczne

Celem pracy licencjackiej nie jest prezentacja wiedzy, lecz zrozumienie problemu. Proces ten obejmuje analizę dowodów i założeń oraz przemyślenie logiki argumentów. Punktem wyjścia jest pytanie – i nie każde pytanie się do tego nadaje.

5.1.1 Jakie pytanie nadaje się na fundament pracy

Dobre pytanie analityczne dotyczy realnego dylematu obecnego w źródłach, a nie wymyślnego na potrzeby pracy. Daje odpowiedź, która nie jest oczywista przed przeprowadzeniem analizy. Sugeruje odpowiedź na tyle złożoną, że wymaga pełnej dyskusji, a nie akapitu. I da się na nie odpowiedzieć dostępnymi źródłami oraz dostępnymi danymi.

5.1.2 Skąd brać pytania: punkty napięcia

Pytania rodzą się w miejscach, gdzie źródła nie są ze sobą zgodne albo gdzie coś zgrzyta. Warto myśleć o nich jak o momentach, w których trzeba się zatrzymać i zastanowić, zanim da się iść dalej. Szukaj:

- stanowiska, które wydaje Ci się nieprzekonujące – zapytaj, czego brakuje albo jak dowody można przeczytać inaczej;
- nieścisłości, luki lub dwuznaczności w dowodach – zapytaj, jak to zmienia rozumienie problemu;
- nieoczekiwanego wniosku, który nie do końca się zgadza – zapytaj, jak autorzy do niego doszli;
- kontrowersji, która wymaga rozstrzygnięcia – zapytaj, jak można ją rozstrzygnąć;
- problemu, który został zignorowany – zapytaj, dlaczego, albo spróbuj go rozwiązać;
- dowodu, który zasługuje na bliższe przyjrzenie się.

5.1.3 Napięcia właściwe dla pracy o oprogramowaniu badawczym

W Twoim temacie napięcia są też innego rodzaju – między metodą a jej użyciem. Artykuł podaje algorytm, ale **nie precyzuje**, co zrobić z brakami danych, wartościami brzegowymi albo przypadkiem zdegenerowanym. Metoda zakłada dane, których w badaniach społecznych zwykle **nie ma** w tej postaci. Istniejąca implementacja daje **inny wynik** niż opis w publikacji. Autorzy pokazują metodę na danych symulowanych i nie mówią, co dzieje się na danych rzeczywistych. Procedura wymaga decyzji parametrycznej, którą publikacja przemilcza, choć wynik od niej zależy.

Każde z tych napięć jest gotowym punktem wyjścia dla pytania badawczego – i jednocześnie decyzją projektową w pakiecie.

5.1.4 Jak formułować

Pytania „jak” i „dlaczego” wymagają więcej analizy niż „kto”, „co”, „gdzie” i „kiedy”. Dobre pytanie podkreśla wzorzec i związek albo sprzeczność i dylemat. Wyznacza zakres argumentacji, pozwalając skupić się na części szerokiego tematu, i może dotyczyć implikacji oraz konsekwencji analizy.

Pytanie prowadzi przez cały proces badawczy i pisanie. Kształtuje strukturę pracy, terminologię i kierunek poszukiwań – dlatego warto poświęcić mu pierwsze tygodnie seminarium, a nie wymyślać je w tydzień przed oddaniem.

5.2 Od pytania do tezy

Pytanie otwiera, teza domyka. Teza to zdanie oznajmujące, z którym da się nie zgodzić. Jeżeli nikt rozsądny nie mógłby zaprzeczyć Twojej tezie, to nie jest teza, tylko stwierdzenie faktu.

Tabela 18. Od pytania do tezy

Pytanie	Teza
Jakie warunki musi spełnić implementacja metody, żeby wynik dał się odtworzyć?	Odtwarzalność wyniku zależy w większym stopniu od jawnego opisu obsługi braków danych niż od precyzji samego algorytmu.
Które założenia metody są najbardziej wrażliwe na jakość danych społecznych?	Metoda traci stabilność przy współliniowości kryteriów wcześniej niż przy brakach danych, co odwraca kolejność zaleceń podawanych w literaturze.

Teza może się zmienić w trakcie pracy. To normalne i nie jest porażką – zmiana tezy pod wpływem wyników jest dowodem uczciwości, o ile praca opisuje wersję finalną, a nie obie naraz.

5.3 Wejście w dyskusję akademicką

Praca ma być głosem w rozmowie, która toczy się bez Ciebie. Są trzy sposoby włączenia się.

5.3.1 Krok w przód

Prezentujesz własną wiedzę i wkład: co zrobiłeś, czego to dowodzi, dlaczego to ważne. Wykazujesz znajomość literatury i podkreślasz, na czym polega unikalność Twojego rozwiązania. To dominujący tryb Wprowadzenia i rozdziału drugiego.

Zacznij od przeglądu istniejących prac, teorii i terminów. Czytaj nie tylko to, co autorzy mówią, ale też **to, co pomijają albo traktują pobieżnie** – tam zwykle leży luka. Następnie wskaż, gdzie dokładnie Twoja praca wpisuje się w te dyskusje. Nie wystarczy powiedzieć, że praca jest z tym związana; trzeba określić, jak Twoje podejście różni się od obecnego rozumienia albo je rozszerza.

5.3.2 Krok w tył

Umieszczasz własną argumentację w szerszym kontekście: jak praca koreluje z innymi badaniami, jak rozszerza albo kwestionuje istniejące ustalenia. To dominujący tryb rozdziału trzeciego i Podsumowania.

Zastanów się też, kto będzie Twoją pracą zainteresowany – inni badacze, praktycy, szersza publiczność. Odbiorca decyduje o języku i o tym, które konsekwencje warto wyeksponować. Praca o oprogramowaniu badawczym ma odbiorcę podwójnego: metodologa, którego interesuje wierność implementacji, i badacza, którego interesuje, czy narzędzie da się użyć.

5.3.3 Kwestionowanie

Podważasz przyjęte założenia i pokazujesz alternatywne perspektywy. Wyjaśniasz, dlaczego te alternatywy są poznawczo ważne, ale niemożliwe do wykorzystania przy Twoim celu, i że stanowią obszar ograniczenia Twojej perspektywy, wykraczający poza Twój problem.

Bądź gotowy na kontrargumenty. To normalne, że recenzent nie zgodzi się z częścią wniosków. Przewidywanie sprzeciwów i odpowiadanie na nie w tekście wzmacnia pracę bardziej niż ich pominięcie.

5.4 Zapożyczanie i rozwijanie teorii

Zidentyfikuj teorie i pojęcia istotne dla tematu. Zwróć uwagę na kluczowe idee, definicje i argumenty – bez tego nie da się ich skutecznie wprowadzić do własnej pracy.

W pracach z zakresu nauki o informacji pojawiają się terminy wymagające definicji, jak metadane, analiza semantyczna, uczenie maszynowe czy odtwarzalność. Wyjaśnij, jak używasz ich **w kontekście swojej pracy** i jakie mają znaczenie dla zrozumienia problemu. Definicja robocza jest lepsza niż definicja słownikowa, o ile powiesz, skąd ją bierzesz.

Następnie rozwiń te idee: jak teorie stosują się do Twojego tematu, co trzeba w nich dostosować. Wreszcie pokaż, w jaki sposób te założenia wpłynęły na Twoje decyzje – na kształt metod, na sposób weryfikacji, na interpretację wyników. Nie chodzi o powierzchowne odniesienia, lecz o widoczne konsekwencje.

W pracy projektowej ten mechanizm ma konkretną postać: **założenie teoretyczne staje się warunkiem wstępnym funkcji**. Jeżeli metoda zakłada nieujemność ocen, w kodzie pojawia się sprawdzenie i komunikat błędu. Rozdział pierwszy i rozdział drugi mają się w tym miejscu spotkać – i recenzent to sprawdzi.

5.5 Czytanie artykułu metodycznego

Artykuł, na którym opierasz pracę, czytasz inaczej niż literaturę przeglądową.

Kolejność. Najpierw abstrakt i wnioski, żeby wiedzieć, po co to powstało. Potem sekcja definicyjna i algorytmiczna – to jest Twój materiał. Sekcje dowodowe i porównawcze na końcu, i tylko w takim zakresie, w jakim wpływają na implementację.

Co wynotować. Dla każdego wzoru: co jest wejściem, co wyjściem, jakiego typu i z jakiej dziedziny jest każdy symbol. Dla każdego kroku: co się dzieje, gdy wejście jest brzegowe. Osobno wypisz zdania zaczynające się od słów „zakładamy”, „przy założeniu”, „o ile” – to jest przyszły kontrakt Twojego pakietu.

Czego szukać między wierszami. Decyzji, które autorzy podjęli, ale ich nie uzasadnili: wartości domyślnych parametrów, sposobu normalizacji, kolejności kroków. Każda taka decyzja jest miejscem, w którym Twoja implementacja musi zająć stanowisko – i w którym narzędzie wspomagające pisanie kodu najczęściej zgaduje.

5.6 Notatki i zarządzanie źródłami

Bibliografię prowadź **od pierwszego dnia**. Odtwarzanie jej po fakcie zajmuje kilka dni i zawsze kończy się brakującym numerem strony.

Praktyka minimalna obejmuje cztery kroki. Utwórz kolekcję w Zotero z podkolekcjami na metodę, dziedzinę i narzędzia. Ustaw automatyczny eksport do pliku bibliograficznego projektu. Przy każdej pozycji zapisz jedno lub dwa zdania o tym, **do czego ta pozycja jest Ci potrzebna w pracy** – nie streszczenie, bo streszczenie jest w abstrakcie. Cytowanie w tekście wstawiaj od razu przy pisaniu akapitu, a nie na końcu rozdziału.

Sprawdzanie metadanych po imporcie jest obowiązkowe: bazy bibliograficzne notorycznie gubią drugie imiona, mylą typ dokumentu i skracają tytuły czasopism. Szczegóły techniczne są w rozdziale 2.

5.7 Styl naukowy

5.7.1 Terminologia

Termin raz wprowadzony ma jedną formę w całej pracy. Jeżeli w rozdziale pierwszym piszesz „wariant”, nie przechodź w rozdziale trzecim na „alternatywę”. Konsekwencja terminologiczna jest osobnym kryterium oceny (Standardy EPI).

Termin obcojęzyczny wprowadzasz raz, z polskim odpowiednikiem i zapisem oryginalnym w nawiasie, dalej używasz jednej formy. Nazw funkcji i pakietów nie odmieniamy – poprzedzamy je rzeczownikiem: funkcja `przygotuj_dane()`, pakiet `ggplot2`.

5.7.2 Akapit

Akapit ma jedną myśl i mieści się na jednej trzeciej strony. Pierwsze zdanie mówi, o czym akapit będzie. Ostatnie wiąże go z następnym. Akapit na całą stronę zawsze da się podzielić i zawsze na tym zyskuje.

5.7.3 Rejestr

Praca naukowa różni się od publicystyki i od dokumentacji technicznej.

Tabela 19. Rejestr wypowiedzi

Zamiast	Napisz
Niesamowicie ważnym zagadnieniem jest...	Zagadnienie ma znaczenie dla..., ponieważ...
Jak wiadomo, media społecznościowe...	Badania nad mediami społecznościowymi wskazują, że...(cytowanie)
Postanowiłem użyć tej metody, bo jest wygodna	Wybrano metodę ze względu na...
Funkcja bierze dane i je przetwarza	Funkcja przyjmuje macierz ocen i zwraca wektor wag

Unikaj nadmiernych skrótów myślowych, wypowiedzi emocjonalnych i publicystycznych, rozbudowanych dygresji oraz treści luźno związanych z tokiem rozumowania. Nie opieraj wniosków na nieujawnionych przesłankach ani na niepotwierdzonych naukowo przekonaniach (Instrukcja ISI).

5.7.4 Precyzja

Tezy, sądy i wnioski formułuj precyzyjnie. Zdanie o tym, że metoda działa lepiej, nie znaczy nic; zdanie o tym, że metoda daje niższy błąd średniokwadratowy przy próbach poniżej stu obserwacji, znaczy. Każde twierdzenie porównawcze wymaga wskazania, względem czego i na jakiej podstawie.

5.8 Rzetelność

5.8.1 Oryginalność

Praca nie może powielać badań przeprowadzonych wcześniej przez inne osoby – z wyjątkiem sytuacji, gdy chodzi o ich weryfikację lub aktualizację. Implementacja opublikowanej metody **jest** dopuszczalna i mieści się w tym wyjątku, o ile praca wnosi własny wkład: rozstrzygnięcie niedopowiedzeń, procedurę weryfikacji, zastosowanie w nowym obszarze.

5.8.2 Kompilacja

Praca ani żaden jej fragment nie może być zestawieniem zapożyczeń – ani dosłownych cytatów, ani fragmentów sparafrazowanych. Zasada obejmuje także tłumaczenia z języka obcego. Parafraza nie zwalnia z cytowania: wyrażenie cudzej myśli własnymi słowami bez wskazania autora jest naruszeniem, nawet jeżeli żadne zdanie nie zostało przepisane.

Dosłowne cytowanie i referowanie cudzych ustaleń ogranicz do minimum koniecznego do ustalenia stanu badań, podjęcia z nimi dyskusji albo osadzenia własnych wyników.

5.8.3 Granica między własnym a cudzym

Wypowiedź musi być tak skonstruowana, a przypisy umieszczone w takich miejscach, żeby jednoznacznie oddzielić własne poglądy od zapożyczonych. **W żadnym miejscu czytelnik nie może mieć wątpliwości, co jest wynikiem Twojej pracy, a co odwołaniem do cudzej.** W praktyce oznacza to zwroty sygnalizujące: cudze wprowadzasz zwrotami w rodzaju „zdaniem”, „jak wykazuje”, „w ujęciu”; własne – zwrotami „w niniejszej pracy przyjęto”, „uzyskane wyniki wskazują”, „na tej podstawie można stwierdzić”.

Zasada nie dotyczy wiedzy powszechnej, danych powszechnie dostępnych i ogólnie określonych metod badawczych. Dotyczy natomiast konkretnych teorii, idei, danych i narzędzi badawczych, które stanowią dorobek konkretnych osób.

5.8.4 Stan badań w układzie problemowym

Dotychczasowy dorobek poddajesz samodzielnej analizie i przedstawiasz w postaci własnych wniosków, uporządkowanych **problemowo, a nie chronologicznie ani alfabetycznie**. Układ docelowy: co już wiemy i co jeszcze wymaga zbadania.

Przegląd, który brzmi jak wyliczanka kolejnych autorów i tego, co napisali, nie jest analizą, tylko listą. Analiza brzmi inaczej: w danej kwestii literatura jest zgodna co do jednego, natomiast rozchodzi się w sprawie drugiego, gdzie jedni autorzy wskazują na to, a inni przeciwnie.

5.9 Redakcja i korekta

Redakcja to osobny etap, a nie przedłużenie pisania. Zaplanuj na nią co najmniej tydzień.

Przeglądy rób po kolei, za każdym razem szukając jednej rzeczy. Najpierw struktura: czy kolejność rozdziałów i sekcji odpowiada logice wyводу. Potem argument: czy każda teza ma uzasadnienie, a każde uzasadnienie prowadzi do tezy. Następnie cytowania: czy każde twierdzenie o cudzym dorobku ma przypis i czy każda pozycja bibliografii ma cytowanie. Dalej terminologia: czy nazwy są konsekwentne w całej pracy. Potem język: zdania, akapity, interpunkcja. Na końcu skład: wiszące wiersze, pływające tabele, przeniesienia, spisy.

Korektę rób na wydruku albo w innym kroju pisma. Czytaj na głos zdania, które wydają się długie; zdanie, którego nie da się przeczytać jednym tchem, wymaga podziału. Przed ostatnim przeglądem odłóż tekst na dwa lub trzy dni – nie ma sposobu, żeby to obejść.

5.10 Harmonogram

Pisanie i budowa pakietu prowadzone są równolegle. Rozdziały powstają w kolejności innej niż kolejność w pracy; jest ona w tabeli 17.

Tabela 20. Co prowadzić od pierwszego dnia

Co	Gdzie	Dlaczego
plik bibliograficzny	katalog bibliografii	odtworzenie po fakcie zajmuje kilka dni
polecenia wstawiające nazwiska	rozdziały	indeks nazwisk powstaje sam
rejestr poleceń dla na-rzędzia	repozytorium pakietu	wymagany w aneksie, nie da się odtworzyć z pamięci

Reguła praktyczna na koniec: **nie pisz akapitu, którego nie umiesz opowiedzieć na głos w trzech zdaniach**. Jeżeli nie umiesz, nie jest to problem pisania, tylko znak, że ta część nie jest jeszcze przemyślana.

6 Wymogi formalne

Zestawienie wymagań obowiązujących pracę licencjacką na kierunku elektroniczne przetwarzanie informacji, z podaniem źródła każdego z nich. Dwa dokumenty Instytutu uzupełniają się i miejscami rozchodzą – przy rozbieżności o **układzie pracy** rozstrzygają Standardy (Standardy EPI), a o **zapisie bibliografii** Instrukcja (Instrukcja ISI). W tabelach skrót S oznacza Standardy, a skrót I – Instrukcję.

6.1 Skład

Tabela 21. Wymogi składu

Wymóg	Źródło	Kto realizuje
Pismo 12 pkt w tekście głównym	S	klasa
Interlinia 1,5	S	klasa
Marginesy 2,5 cm z każdej strony	S, I	klasa
Przy oprawie margines wewnętrzny szerszy kosztem zewnętrznego, nie mniej niż 1,5 cm	I	opcja klasy
Tytuły rozdziałów tym samym krojem, większe i wytłuszczone	S	klasa
Tytuły rozdziałów 18 pkt, wyśrodkowane	I	klasa
Tytuły podrozdziałów 14 pkt, wytłuszczone, do lewej	I	klasa
Numeracja rozdziałów wielopoziomowa	S, I	klasa
Wstęp i Zakończenie bez numeracji, ale w spisie treści	I	klasa
Każdy element ze spisu treści od nowej strony	S, I	klasa
Wszystkie strony numerowane	S	klasa
Numer strony w prawym dolnym rogu	I	klasa
Strona tytułowa liczona, bez numeru	S, I	klasa
Tekst wyjustowany	S, I	klasa

Wszystkie pozycje realizuje klasa dokumentu bez Twojego udziału. Nie zmieniaj ustawień składu w preambule – rozbieżność z wymogami obciąża pracę, a nie klasę.

6.2 Objętość

Tekst od pierwszej strony Wprowadzenia do ostatniej strony Podsumowania nie może przekraczać **czterdziestu stron znormalizowanych, czyli 72 000 znaków ze spacjami**

(Instrukcja ISI). Do limitu nie wliczają się: strona tytułowa, strony informacyjne, spis treści, wykaz źródeł, bibliografia, spisy, indeks i aneksy.

Lokalnie. Kontrolę wykonuje skrypt `tools/policz_znaki.R`, uruchamiany z katalogu wzoru poleceniem `Rscript tools/policz_znaki.R`. Liczy znaki po usunięciu poleceń, listin-
gów, wzorów i tabel, więc daje oszacowanie od góry. Wymaga zainstalowanego R.

Na Overleaf. R-a tam nie ma i skrypt się nie uruchomi. Zgrubną kontrolę daje wbudowane liczenie słów z menu projektu: strona znormalizowana to około 250 słów tekstu polskiego, więc limit odpowiada mniej więcej dziesięciu tysiącom słów. Kontrolę rozstrzygającą robisz na gotowym dokumencie: zaznacz tekst od pierwszej strony Wprowadzenia do ostatniej strony Podsumowania, wklej do edytora tekstu i odczytaj liczbę znaków ze spacjami.

6.3 Struktura

Kolejność elementów we wzorze łączy wymagania obu dokumentów: strona tytułowa, opcjonalna strona informacyjna z abstraktem i słowami kluczowymi, spis treści, Wprowadzenie, rozdziały numerowane, Podsumowanie, wykaz źródeł, bibliografia, spis ilustracji, indeks nazwisk, aneksy.

Standardy opisują treść rozdziałów zasadniczych jako logikę aplikacji – schemat ogólny, moduły, opis interfejsu ze zrzutami ekranu – oraz implementację, czyli format danych, strukturę bazy, sposób realizacji modułów, biblioteki zewnętrzne i narzędzia. Wzór realizuje to w trzech rozdziałach.

6.3.1 Strona informacyjna

Wymóg Instrukcji, włączany opcją klasy. Zawiera opis bibliograficzny pracy w postaci obejmującej nazwisko i imię autora, rok obrony, tytuł, oznaczenie rodzaju pracy, promotora, miejsce, jednostkę i liczbę stron. Dalej abstrakt liczący **od 1000 do 1500 znaków ze spacjami**, przedstawiający cel, przedmiot, metody i kluczowe wyniki, napisany językiem skondensowanym i bez ogólników. Na końcu **około pięciu słów kluczowych** w porządku alfabetycznym, rozdzielonych przecinkami.

Strona angielska powstaje automatycznie po wypełnieniu angielskich metadanych i zawiera dokładnie te same informacje w tym samym układzie.

6.4 Cytowanie

Standardy wymieniają pięć sytuacji, w których wskazanie źródła jest obowiązkowe:

1. cytat bezpośredni, czyli dosłownie przytoczone cudze słowa;

2. graficzna forma cytatu – cudze tabele, rysunki, zestawienia;
3. cytat znany pośrednio, z tekstu jeszcze innego autora;
4. zebrane przez kogoś informacje: ankiety, dane statystyczne, dokumenty, materiały ze stron internetowych;
5. cudza teza, argument, opinia, idea, interpretacja, unikalne ujęcie tematu.

Każde przytoczenie umieszczasz w cudzysłowie polskim, a cytat w cytacie w cudzysłowie wewnętrznym. Wewnątrz cytatu obowiązuje pisownia cytowanego autora; zmianę odnotowujesz w nawiasie kwadratowym po cudzysłowie zamykającym. Tłumaczenie własne oznaczasz w nawiasie kwadratowym, a cytat w oryginale przywołujesz w przypisie dolnym. Fragment powyżej dwóch lub trzech zdań wyodrębniasz graficznie.

Przytoczenie musi być **wkomponowane w tekst**: wprowadzone lub domknięte komentarzem, który sygnalizuje obecność cudzego słowa. Samo wklejenie z cytowaniem nie wystarcza. Przy powoływaniu się na cudzą opinię obowiązuje, niezależnie od cytowania, wprowadzenie z informacją, do kogo dany pogląd należy.

Brak zasygnalizowania w tekście, że posługujesz się cudzym słowem, oznacza naruszenie prawa (Dz.U. 2022 poz. 2509). Brak odpowiednio opisanych źródeł może być powodem dyskwalifikacji pracy.

6.5 Przypisy i bibliografia

Obowiązuje system nawiasowy, tak zwany harwardzki: nazwisko, rok, w razie potrzeby strona. Postacie cytowań i odpowiadające im polecenia są zestawione w tabeli 10. Skracanie listy autorów, dodawanie inicjału i sufiksów rocznika wykonuje styl automatycznie – Twoim zadaniem jest poprawny wpis w pliku bibliograficznym.

Do przypisów dolnych trafiają wyłącznie przypisy dygresyjne, których należy unikać, oraz oryginalne brzmienie tłumaczonych cytatów.

Zasada wiążąca bibliografię z tekstem: **bibliografia zawiera wyłącznie pozycje faktycznie wykorzystane, do których odsyłają cytowania w tekście**. Pozycje dotyczące tematu, ale niewykorzystane, do bibliografii nie trafiają. Muszą się w niej znaleźć pozycje obcojęzyczne.

6.6 Materiał ilustracyjny

Środowiska wzoru realizują tytuł, numerację i miejsce na źródło automatycznie.

Prawa do ilustracji: bez zgody autora można wykorzystać materiały z domeny publicznej, na licencji Creative Commons lub podobnej oraz zrzuty ekranu z publicznie dostępnych

Tabela 22. Wymogi wobec materiału ilustracyjnego

Wymóg	Źródło
Numeracja ciągła przez cały tekst, osobna dla każdego typu	I
Tytuł nad ilustracją	I
Źródło pod ilustracją	I
Oznaczenie opracowania własnego, także na podstawie cudzego materiału	I
Pełny opis bibliograficzny dla materiału przejętego	I
Interfejs aplikacji zilustrowany zrzutami ekranu	S

źródeł elektronicznych. Pozostałe wymagają pisemnej zgody właściciela autorskich praw majątkowych.

6.7 Zapis w tekście

Tabela 23. Zapis elementów w tekście

Element	Zapis	Polecenie
tytuł czasopisma, książki, zasobu sieciowego	kursywa bez cudzysłowu	<code>\termin</code>
wyraz obcojęzyczny	kursywa	<code>\termin</code>
wyróżnienie treściowe	wytluszczenie	<code>\wyznienienie</code>
cytat	cudzysłów polski	<code>\enquote</code>
nazwa funkcji	pismo maszynowe	<code>\fun</code>
nazwa argumentu	pismo maszynowe	<code>\argument</code>
nazwa pakietu	pismo maszynowe	<code>\pkg</code>
nazwa pliku	pismo maszynowe	<code>\plik</code>

6.8 Wymogi projektu dyplomowego

Poza pracą pisemną Standardy wymagają samodzielnie zaprojektowanej i wykonanej aplikacji z interfejsem internetowym. Zaliczenie seminarium następuje dopiero po przyjęciu projektu (Standardy EPI; Prawo o szkolnictwie wyższym).

Aplikacja nastawiona na prezentację informacji musi robić więcej niż wyświetlanie zawartości bazy danych: konieczny jest niestandardowy pomysł prezentacji, dostosowany projekt graficzny i mechanizm interakcji z użytkownikiem.

W serwisie obowiązkowa jest nota z załącznika do Standardów, umieszczona w zakładce o serwisie. Podaje ona autora serwisu, informację, że serwis stanowi integralną część pracy licencjackiej na kierunku elektroniczne przetwarzanie informacji, oraz promotora i jednostkę. Ta sama nota trafia do aneksu pracy.

6.9 Kryteria oceny

Standardy wymieniają wprost, co podlega ocenie promotora i recenzenta: zgodność treści pracy z tematem, strukturę pracy, znajomość stanu badań lub praktyki w zakresie wyznaczonym przez tematykę, poprawność językową i terminologiczną, metodologię projektu, zawartość merytoryczną, poprawność działania oraz estetykę aplikacji, a także dobór i wykorzystanie piśmiennictwa. Recenzje trafiają do Archiwum Prac.

6.10 Lista kontrolna przed oddaniem

Kompilacja i objętość

- ☐ Pełna kompilacja kończy się bez błędów
- ☐ Brak komunikatów o nierozwiązanych cytowaniach i odsyłaczach
- ☐ Objętość mieści się w limicie
- ☐ Listingi nie zawierają znaków spoza ASCII

Treść

- ☐ Tytuł pracy odpowiada zawartości i zawiera zagadnienie ogólniejsze
- ☐ Cel z Wprowadzenia domknięty w Podsumowaniu
- ☐ Perspektywy rozwoju aplikacji opisane konkretnie
- ☐ Wszystkie przytoczenia wprowadzone komentarzem w tekście
- ☐ Własne ustalenia oddzielone od cudzych na poziomie zdania

Bibliografia i źródła

- ☐ Każda pozycja bibliografii ma cytowanie w tekście
- ☐ Każde cytowanie ma pozycję w bibliografii
- ☐ W bibliografii są pozycje obcojęzyczne
- ☐ Wykaz źródeł obejmuje wszystkie użyte pakiety
- ☐ Adresy sieciowe mają datę odczytu

Elementy końcowe

- ☐ Spis ilustracji zgodny z zawartością pracy
- ☐ Indeks nazwisk obejmuje nazwiska z tekstu i bibliografii
- ☐ Aneksy powiązane z tekstem odsyłaczami

- ☐ Aneks z metadanymi oprogramowania wypełniony
- ☐ Aneks z notą serwisu zgodny z treścią na stronie projektu
- ☐ Oświadczenie o wykorzystaniu sztucznej inteligencji wypełnione, jeżeli dotyczy
- ☐ Wykaz poleceń wydanych narzędziu programistycznemu kompletny

Projekt

- ☐ Strona projektu działa pod adresem podanym na stronie tytułowej
- ☐ Nota z załącznika obecna w serwisie
- ☐ Repozytorium publiczne i zgodne z adresem ze strony tytułowej
- ☐ Sprawdzenie pakietu przechodzi na trzech systemach operacyjnych

Redakcja

- ☐ Korekta całości po wydruku, nie na ekranie
- ☐ Tekst wyjustowany, bez wiszących wierszy
- ☐ Konsekwentna terminologia w całej pracy
- ☐ Wyszukanie uwag promotora w plikach źródłowych nie daje wyników
- ☐ Usunięte własne komentarze robocze i wskazówki wzoru

7 Praca z agentem programistycznym

Agent programistyczny, nazywany też agentem SI, to program oparty na dużym modelu językowym, uruchamiany z terminala w katalogu projektu. Dostaje od Ciebie cel opisany słowami, po czym sam czyta pliki projektu, pisze i zmienia kod, uruchamia testy, czyta ich wyniki i poprawia to, co nie przeszło. Pracuje w pętli i sam decyduje, które pliki otworzyć.

W rozdziale opisana jest metoda budowy pakietu z użyciem takiego agenta. Granice dozwolonego użycia i sposób ich dokumentowania są w rozdziale 8; tutaj chodzi o to, **jak** pracować, żeby powstało narzędzie badawcze, za które da się odpowiadać.

Nazywamy rzecz po imieniu, bo seminarium wymaga jawności użycia. Praca, która opisuje agenta jako „narzędzie wspomagające”, zaciera to, co recenzent ma ocenić: gdzie kończy się wkład autora, a gdzie zaczyna wynik działania programu.

Jedno zastrzeżenie na wstępie: **pakiet budujesz zawsze lokalnie**, niezależnie od tego, gdzie piszesz pracę. Overleaf składa dokumenty i nie uruchamia R-a, więc kod, testy i sprawdzenie pakietu dzieją się u Ciebie na komputerze. Podział na dwie ścieżki z podrozdziału `efsec:ścieżki` dotyczy tekstu pracy, nie projektu.

Punkt wyjścia jest taki: implementacja algorytmu z artykułu metodycznego to zadanie, w którym trudność nie leży w składni języka, tylko w zrozumieniu metody. Agent zdejmuje z Ciebie pierwsze i **nie zdejmuje drugiego**. Twoja rola przesuwą się z pisania linii kodu na projektowanie kontraktu i sprawdzanie, czy wynik ten kontrakt spełnia. To trudniejsza rola, nie łatwiejsza.

7.1 Środowisko

7.1.1 Positron

Positron jest środowiskiem pracy zbudowanym wokół języka R. Ma konsolę R, podgląd zmiennych, podgląd ramek danych i wbudowany terminal, więc pakiet, wykresy i tekst powstają w jednym miejscu. Jest zbudowany na tej samej podstawie co popularne edytory kodu, więc rozszerzenia i skróty klawiszowe działają tak, jak można się spodziewać.

Instalacja obejmuje trzy kroki: samo środowisko, aktualną wersję R oraz rozszerzenie do LaTeX-a, jeżeli chcesz pisać w nim także pracę. Po pierwszym uruchomieniu sprawdź w konsoli, czy widzi właściwą wersję R:

```
R.version.string  
.libPaths()
```


7.1.2 Wybór agenta

Rozwiązanie domyślne w seminarium to agent dostępny w ramach pakietu studenckiego platformy, na której trzymasz repozytorium, bo nie wymaga własnego abonamentu. Programów tej klasy jest kilka: Copilot CLI, Codex CLI, Claude Code CLI, OpenCode, HermesAgent. Różnią się modelem, limitami i sposobem rozliczenia, natomiast metoda pracy opisana w tym rozdziale jest dla wszystkich taka sama.

Agenta uruchamiasz z terminala w katalogu pakietu, dzięki czemu widzi cały projekt: pliki źródłowe, testy i komunikaty z ich uruchomienia.

Warunkiem sensownej pracy jest to, żeby agent **sam uruchamiał testy**. Bez tego dostajesz kod, który wygląda poprawnie, i musisz sprawdzać go ręcznie. Z tym dostajesz kod, który przechodzi testy napisane przez Ciebie wcześniej.

Limity bywają miesięczne i potrafią się wyczerpać w najgorszym momencie. Przejście na inny program z powyższej listy niczego nie zmienia w metodzie pracy; w rejestrze poleceń odnotowujesz tylko, którego agenta użyłeś do którego zadania.

7.1.3 Podpowiadanie w edytorze to co innego

Uzupełnianie kodu w edytorze podpowiada kolejny wiersz na podstawie tego, co masz na ekranie. Decyzję podejmujesz Ty, wiersz po wierszu, i każdą propozycję widzisz przed przyjęciem. Agent działa inaczej: dostaje cel, sam wybiera pliki, wprowadza zmiany w kilku miejscach naraz i sam sprawdza wynik.

Rozróżnienie ma skutek praktyczny dla rejestru poleceń. Podpowiedzi w edytorze nie rejestrujesz, bo nie ma czego: nie ma polecenia ani wyodrębnionej zmiany. Każde polecenie wydane agentowi rejestrujesz. Granica przebiega tam, gdzie **istnieje polecenie, które da się przytoczyć**.

7.1.4 Konfiguracja repozytorium

W katalogu głównym swojego pakietu umieszczasz plik z instrukcjami dla agenta. Jest to zwykły plik tekstowy, który agent czyta przy każdym uruchomieniu. Gotowy do uzupełnienia znajdziesz w materiałach seminarium.

Instrukcje robią trzy rzeczy. Ustalają konwencje projektu, żeby nie trzeba było ich powtarzać przy każdym poleceniu. Zakazują tego, co w tym projekcie jest niedopuszczalne, na przykład dopisywania zależności do pliku opisu pakietu bez uzgodnienia. Wymuszają uruchomienie testów po każdej zmianie.

Rozgraniczenie odpowiedzialności zapisz wprost. Pliki, których agent nie zmienia bez Twojej decyzji, to: specyfikacja, testy, procedura generowania danych oraz plik opisu pakietu. Są to miejsca, w których zapisane są Twoje ustalenia metodyczne. Kod obliczeniowy i dokumentacja funkcji są obszarem, w którym agent pracuje.

7.2 Kolejność pracy

7.2.1 Cztery kroki

Obowiązująca kolejność to specyfikacja, testy, implementacja, dokumentacja. Nie jest to preferencja stylistyczna, tylko jedyny układ, w którym da się stwierdzić, czy kod jest poprawny.

Krok pierwszy: specyfikacja. Czytasz artykuł źródłowy i spisujesz kontrakt: co wchodzi, w jakich dziedzinach, jakie warunki muszą zachodzić, co wychodzi, jakie własności muszą być spełnione. Szablon dokumentu znajdziesz w materiałach seminarium. Ten dokument piszesz sam, bez agenta, bo powstaje z lektury, a nie z kodu.

Krok drugi: testy. Z przypadków wypisanych w specyfikacji robisz testy. Każdy przypadek analityczny ma wynik policzony ręcznie ze wzoru. Testy piszesz **przed** implementacją i uruchamiasz je: mają zawieść, bo funkcji jeszcze nie ma. To brzmi absurdalnie, dopóki nie zobaczysz, jak łatwo napisać test, który przechodzi zawsze.

Krok trzeci: implementacja. Dopiero teraz uruchamiasz agenta, dając mu specyfikację i testy jako kontekst. Polecenie brzmi: zaimplementuj funkcję tak, żeby przechodziła te testy, nie zmieniając testów.

Krok czwarty: dokumentacja. Bloki dokumentacyjne powstają ze specyfikacji, a nie z kodu. Opis argumentu ma mówić, co argument znaczy, a nie jakiego jest typu.

7.2.2 Dlaczego odwrotna kolejność zawodzi

Kiedy najpierw powstaje kod, a testy potem, testy powstają **z obserwacji zachowania kodu**. Jeżeli implementacja zawiera błąd, test ten błąd utrwała. Sprawdzasz wtedy, czy kod się nie zmienił, a nie czy liczy poprawnie.

To rozróżnienie jest sednem całej metody i wraca w rozdziale 4 jako wymóg wobec rozdziału drugiego pracy. Test, którego oczekiwana wartość pochodzi z uruchomienia własnego kodu, nie jest dowodem poprawności.

7.3 Pętla naprawcza

Po wydaniu polecenia agent pracuje w cyklu: pisze kod, uruchamia testy, czyta komunikat błędu, poprawia, uruchamia ponownie. Cykl kończy się, gdy testy przechodzą.

Twoja rola w tym cyklu nie polega na czekaniu. Polega na obserwowaniu, **co agent zmienia**, żeby doprowadzić do przejścia testów.

7.3.1 Kiedy przerwać

Są trzy sytuacje, w których pętlę trzeba zatrzymać, bo dalsze krążenie tylko pogarsza kod.

Agent zmienia test zamiast kodu. Test jest zapisem tego, co metoda ma robić. Zmiana testu, żeby przechodził, jest zmianą definicji problemu. Jeżeli test jest błędny, poprawiasz go sam, po sprawdzeniu w artykule, i odnotowujesz to w specyfikacji.

Agent osłabia warunek wstępny. Sprawdzenie, które zatrzymywało wykonanie przy danych spoza dziedziny, znika albo zamienia się w ostrzeżenie. Funkcja zaczyna zwracać wynik tam, gdzie powinna odmówić działania. To najgroźniejszy rodzaj regresu, bo testy przechodzą, a pakiet staje się niebezpieczny.

Agent dodaje zależność. Zamiast rozwiązać problem, sięga po gotową funkcję z pakietu, którego w projekcie nie ma. Każda zależność zwiększa ryzyko, że pakiet przestanie się instalować, i wymaga Twojej świadomej decyzji.

7.3.2 Miejsca, w których agent zgaduje

Agent nie zna artykułu, który czytasz. Zna wzorce z kodu, który widział wcześniej. Tam, gdzie Twoja metoda odbiega od wzorca, dostaniesz rozwiązanie typowe, a nie właściwe.

Tabela 24. Typowe miejsca rozejścia się implementacji z metodą

Sytuacja	Co sprawdzić
Metoda wymaga określonej normalizacji	czy zastosowano tę z artykułu, a nie najpopularniejszą
Artykuł podaje wartość domyślną parametru	czy nie została zastąpiona wartością typową dla innej metody
Wzór zawiera logarytm albo dzielenie	jak obsłużono zero i wartości ujemne
Metoda zakłada określoną kolejność kroków	czy kroków nie przestawiono dla wygody
Wynik ma być obiektem określonej klasy	czy nie zwrócono zwykłej ramki danych
Występują remisy w porządkowaniu	czy sposób ich rozstrzygania jest ten sam co w artykule

Każde z tych miejsc powinno mieć wcześniej wpis w specyfikacji, w tabeli niedopowiedzeń artykułu. Jeżeli go ma, wychwycenie rozejścia zajmuje chwilę. Jeżeli nie ma, wychwycisz je dopiero wtedy, gdy wynik nie zgodzi się z publikacją.

7.4 Szablony poleceń

Polecenie wydane agentowi jest tekstem, który warto przygotować, a nie improwizować. Cztery poniższe schematy pokrywają większość pracy nad pakietem. Wersje gotowe do uzupełnienia znajdziesz w materiałach seminarium.

7.4.1 Wzór na kod

Najczęstsze polecenie w tym seminarium. Kluczowe jest podanie wzoru **razem z objaśnieniem symboli** oraz wskazanie, że wynik ma być liczony na wektorach, a nie w pętli.

Zaimplementuj funkcje <nazwa> w pliku R/<plik>.R.

Wzor:

<wzor w notacji z artykułu>

gdzie:

<objasnienie kazdego symbolu wraz z dziedzina>

Wejscie: <typ, ksztalt, dziedzina>

Wyjscie: <typ i znaczenie>

Wymagania:

- operacje na wektorach zamiast petli po wierszach,
- warunki wstepne sprawdzane na poczatku, z komunikatem po polsku,
- bez nowych zaleznosci,
- testy w tests/testthat/test-<nazwa>.R musza przejsc bez zmian.

Po napisaniu uruchom devtools::test() i pokaz wynik.

7.4.2 Dane, na których metoda nie ma sensu

Polecenie do budowy testów odporności. Sedno polega na tym, żeby prosić o **dane**, a nie o testy: testy piszesz sam, bo to Ty decydujesz, jaki wynik jest poprawny.

Wypisz dziesiec zestawow danych wejsciovych dla funkcji <nazwa>, przy ktorych metoda opisana w SPEC.md nie ma sensu albo dziala na granicy stosowalnosci.

Dla kazdego podaj:

- dane w postaci wywolania R,
- ktory warunek wstepny narusza,
- co powinno sie stac: blad, ostrzezenie czy poprawny wynik.

Nie pisz testow. Nie zmieniaj kodu funkcji.

7.4.3 Diagnostyka zamiast zgadywania

Gdy obliczenie nie zbiega albo daje wynik spoza oczekiwanego zakresu, agent ma sklonnosc do zmieniania kodu na chybil trafil. To polecenie przerywa taki cykl.

Funkcja <nazwa> zwraca <opis objawu> dla danych <opis>.

Nie zmieniaj kodu. Wypisz wartosci posrednie po kazdym kroku algorytmu opisanego w SPEC.md i wskaz krok, w ktorym wynik przestaje odpowiadac oczekiwaniu. Podaj wartosci, ktore o tym swiadcza.

7.4.4 Dokumentacja ze specyfikacji

Napisz bloki dokumentacyjne roxygen2 dla funkcji eksportowanych w pliku R/<plik>.R, na podstawie SPEC.md, a nie na podstawie kodu.

Wymagania:

- opis argumentu mowi, co argument znaczy, nie jakiego jest typu,
- sekcja o wartosci zwracanej opisuje strukture wyniku,
- przyklad wykonuje sie ponizej piatej sekundy i korzysta z danych pakietu,
- odwołanie do artykulu zrodlowego w sekcji references,
- tekst po polsku, identyfikatory bez znakow diakrytycznych.

7.5 Rejestr poleceń

Wykaz poleceń jest wymaganym aneksem pracy, ale prowadzi się go **od pierwszego dnia**, a nie odtwarza z pamięci przed oddaniem. Odtworzenie jest niewykonalne: po pół roku nie pamięta się, które polecenie doprowadziło do której funkcji.

Rejestr jest plikiem w repozytorium pakietu. Dla każdego polecenia zapisujesz pięć rzeczy: użytego agenta, cel, treść polecenia, co zmieniłeś w otrzymanym wyniku oraz jak sprawdziłeś poprawność. Trzecia i czwarta kolumna są najważniejsze, bo to one pokazują Twój wkład.

Tabela 25. Wpis w rejestrze poleceń

Pole	Treść
Agent	nazwa i wersja użytego programu
Cel	Implementacja wyznaczania wag metodą entropii
Polecenie	pełna treść wydanego polecenia
Zmiany własne	Dodano obsługę kolumny o zerowej sumie, której polecenie nie obejmowało; zmieniono komunikat błędu na wskazujący nazwę kryterium
Weryfikacja	Przypadek analityczny z macierzą dwa na dwa policzony ręcznie; sprawdzono niezmienniczość na przeskalowanie kolumny

Rejestr ma jeszcze jedno zastosowanie, ważniejsze od formalnego. Wpis, w którym kolumna „zmiany własne” jest pusta, oznacza kod, którego nie sprawdziłeś. Przed obroną warto przejrzeć rejestr pod tym kątem.

7.6 Co robisz sam

Lista zamykająca rozdział. Poniższe czynności nie są zadaniem dla agenta, bo w każdej z nich rozstrzyga rozumienie metody, a nie znajomość języka.

- lektura artykułu źródłowego i spisanie specyfikacji;
- rozstrzygnięcie niedopowiedzeń artykułu wraz z uzasadnieniem;
- wyliczenie ręczne przypadków analitycznych;
- decyzja o obsłudze braków danych i wartości brzegowych;
- projekt procedury generowania danych;
- decyzja o każdej zależności pakietu;
- interpretacja wyników w studium przypadku;
- cały tekst pracy.

Sprawdzian na koniec: **musisz umieć objaśnić każdą linię kodu, którą oddajesz.** Nie chodzi o pamiętanie składni, tylko o umiejętność powiedzenia, co ta linia liczy i dlaczego akurat tak. Linia, której nie umiesz objaśnić, jest linią, której w Twoim pakiecie nie powinno być.

8 Jawność i odpowiedzialność

Praca dyplomowa jest z ustawy **samodzielnym opracowaniem** zagadnienia, prezentującym wiedzę i umiejętności studenta oraz umiejętność samodzielnego analizowania i wnioskowania (Prawo o szkolnictwie wyższym). Ten wymóg nie zmienia się od tego, że część kodu powstała z pomocą agenta. Zmienia się natomiast to, co musisz o tej pomocy powiedzieć.

W rozdziale opisane są granice dozwolonego użycia, sposób jego dokumentowania i konsekwencje przekroczenia granic. Metoda pracy jest w rozdziale 7.

8.1 Dwa różne obszary

Zasady dla kodu i dla tekstu są **różne** i mylenie ich jest najczęstszym źródłem kłopotów.

8.1.1 Kod aplikacji

Użycie agenta przy budowie pakietu jest **dozwolone** i stanowi element metodyki, którą praca opisuje. Warunki są trzy.

Jawność: każde polecenie dotyczące budowy aplikacji trafia do rejestru, a rejestr do aneksu pracy. Odpowiedzialność: za poprawność kodu odpowiadasz Ty, niezależnie od tego, kto napisał którą linię. Zrozumienie: musisz umieć objaśnić każdą linię, którą oddajesz, i obronić przyjęte rozwiązania.

Sposób, w jaki agent pomógł i w jakich miejscach się pomylił, jest **materiałem do pracy**, a nie wstydliwym szczegółem. Rozdział drugi zawiera krótką analizę: gdzie specyfikacja wychwyciła rozejście się implementacji z metodą i jak to wykryłeś. Dla recenzenta jest to mocniejszy dowód zrozumienia niż bezbłędny kod.

8.1.2 Tekst pracy

Tu zakres jest znacznie węższy. Dozwolone jest **poprawianie tekstu, który sam napisałeś**: korekta językowa, uporządkowanie składni zdania, sprawdzenie interpunkcji, pomoc przy składzie wzorów.

Niedozwolone jest **generowanie tekstu**. Żaden akapit pracy nie może powstać przez wygenerowanie i przyjęcie. Powód nie jest formalny, lecz merytoryczny: praca ma być zapisem Twojego rozumowania, a rozumowanie, którego nie przeprowadziłeś, nie jest Twoje.

8.1.3 Bezwzględny zakaz

Jest jedna kategoria, w której użycie agenta jest wykluczone bez wyjątków: **cytowania, opisy bibliograficzne i treść przypisów**.

Powód jest praktyczny. Agenci wytwarzają opisy bibliograficzne wyglądające poprawnie, ale odsyłające do publikacji, które nie istnieją albo mają inne dane. Nazwiska, tytuły, roczniki i numery stron układają się w wiarygodną całość, której nie da się zweryfikować, bo nie ma czego weryfikować.

Konsekwencje takiego wpisu w pracy dyplomowej są poważne. Brak odpowiednio opisanych źródeł może być powodem dyskwalifikacji pracy (Standardy EPI), a cytowanie nieistniejącej publikacji jest czymś więcej niż brakiem opisu.

Każdą pozycję bibliografii masz w ręku: przeczytaną, z ustalonym identyfikatorem cyfrowym, wpisaną do menedżera bibliografii. Sposób prowadzenia jest w rozdziale 2.

Tabela 26. Zakres dozwolonego użycia

Czynność	Ocena
Implementacja funkcji na podstawie specyfikacji	dozwolone, wymaga wpisu w rejestrze
Generowanie danych złośliwych do testów odporności	dozwolone, wymaga wpisu w rejestrze
Diagnostyka błędu obliczeniowego	dozwolone, wymaga wpisu w rejestrze
Bloki dokumentacyjne funkcji	dozwolone, wymaga wpisu w rejestrze
Korekta językowa własnego akapitu	dozwolone, wpisz do oświadczenia
Pomoc przy składzie wzoru	dozwolone, wpisz do oświadczenia
Napisanie akapitu pracy	niedozwolone
Streszczenie artykułu, którego nie przeczytałeś	niedozwolone
Sformułowanie wniosków z wyników	niedozwolone
Wytworzenie cytowania albo opisu bibliograficznego	wykluczone bez wyjątków

8.2 Oświadczenie

Oświadczenie umieszcza się w aneksie pracy. Nie jest wymagane przy drobnych pracach redakcyjnych, ale takie kwestie jak edytowanie równań, poprawa stylistyczna tekstu czy korekty błędów w kodzie warto w nim wpisać.

Podczas przygotowywania niniejszej pracy korzystałem/am z narzędzia [...] w celu [...]. Po użyciu tego narzędzia dokonałem/am krytycznego przeglądu i edycji treści. Oświadczam, że wygenerowane fragmenty zostały odpowiednio oznaczone, a SI służyła mi wyłącznie jako asystent, nie zaś jako źródło wiedzy. Ponoszę pełną odpowiedzialność za finalny kształt i poprawność przedłożonych analiz.

Pole „w celu” wypełnia się konkretnie. Zapis „do pomocy przy pracy” nic nie znaczy. Zapis „do implementacji funkcji obliczeniowych na podstawie specyfikacji oraz do korekty językowej tekstu własnego” mówi dokładnie, co się działo.

Gotowy do wypełnienia aneks znajduje się we wzorze pracy.

8.3 Aneks z wykazem poleceń

W oświadczeniu podajesz, **że** korzystałeś. W aneksie – **jak**. Dotyczy wyłącznie budowy aplikacji; poleceń dotyczących korekty językowej się nie wykazuje.

Wykaz powstaje z rejestru prowadzonego w repozytorium pakietu, opisanego w podrozdziale 7.5. Do aneksu przenosisz polecenia istotne, a nie wszystkie: te, które doprowadziły do powstania funkcji publicznych, oraz te, przy których musiałeś poprawić otrzymany wynik.

Kolumna „zmiany własne” jest w tym aneksie najważniejsza. Widać w niej, gdzie kończy się agent, a zaczynasz Ty. Aneks, w którym ta kolumna jest wszędzie pusta, świadczy przeciwko pracy.

8.4 Przygotowanie do obrony

Komisja może zapytać o dowolny fragment kodu. Przygotowanie polega na przejściu pakietu funkcja po funkcji i odpowiedzeniu sobie na trzy pytania.

Co ta funkcja liczy i który krok algorytmu ze specyfikacji realizuje? Dlaczego liczy to akurat w ten sposób, skoro dałoby się inaczej? Co się stanie, gdy podać jej dane, których się nie spodziewa?

Jeżeli na którekolwiek pytanie nie umiesz odpowiedzieć, masz w pakiecie kod, którego nie rozumiesz. Zostało jeszcze dość czasu, żeby to naprawić: przeczytać ponownie odpowiedni fragment artykułu, uprościć funkcję albo napisać ją od nowa.

8.5 Granica, której nie przekraczamy

Na koniec zdanie, które warto sobie powtórzyć przed każdym poleceniem wydanym agentowi.

Agent może napisać kod, który przechodzi Twoje testy. Nie przeczyta za Ciebie artykułu, nie rozstrzygnie niedopowiedzenia metody, nie zdecyduje, co zrobić z brakami danych, ani nie wyciągnie wniosku z wyników. **Te rzeczy są pracą dyplomową.** Reszta jest jej zapisem.

Bibliografia

- PN-ISO 690: 2012. Informacja i dokumentacja – Wytyczne opracowania przypisów bibliograficznych i powołań na zasoby informacji.
- R Core Team (2026). R: A Language and Environment for Statistical Computing. R Foundation for Statistical Computing. <https://www.R-project.org/> (accessed: 7.09.2026).
- Rada Instytutu Studiów Informacyjnych UJ (2022). Prace licencjackie. Instrukcja Instytutu Studiów Informacyjnych UJ. Instytut Studiów Informacyjnych UJ. dokument wewnętrzny, przyjęty 21.10.2015, zmieniony 24.11.2021 i 27.04.2022.
- Rada Instytutu Studiów Informacyjnych UJ (2025). Standardy prac dyplomowych na kierunku elektroniczne przetwarzanie informacji. Instytut Studiów Informacyjnych UJ. dokument wewnętrzny, przyjęty 31.05.2023, zmieniony 11.09.2024 i 29.01.2025.
- Ustawa o prawie autorskim i prawach pokrewnych (2022). tekst jednolity, Dz.U. 2022 poz. 2509.
- Ustawa Prawo o szkolnictwie wyższym i nauce (2018). z późniejszymi zmianami, art. 76 pkt 2.

Spis ilustracji

Tabele

1	Zawartość przewodnika	8
2	Zapis typograficzny	11
3	Konwencja przedrostków w etykietach	12
4	Polecenia semantyczne wzoru	12
5	Zapis matematyczny	15
6	Najczęstsze błędy kompilacji	18
7	Budowa klucza cytowania	20
8	Pola wpisu bibliograficznego	23
9	Typy dokumentów w Zotero i we wpisie bibliograficznym	24
10	Postacie cytowania	25
11	Ustawienia projektu na Overleaf	28
12	Problemy w pracy na Overleaf	32
13	Błędy we wprowadzeniu	34
14	Błędy w rozdziale metodycznym	36
15	Błędy w rozdziale inżynierskim	37
16	Błędy w studium przypadku	39
17	Kolejność pisania rozdziałów	41
18	Od pytania do tezy	43
19	Rejestr wypowiedzi	46
20	Co prowadzić od pierwszego dnia	48
21	Wymogi składu	50
22	Wymogi wobec materiału ilustracyjnego	53
23	Zapis elementów w tekście	53
24	Typowe miejsca rozejścia się implementacji z metodą	59
25	Wpis w rejestrze poleceń	61
26	Zakres dozwolonego użycia	64

Rysunki

Spis wykresów